



Automatic Repair of Quantum Programs via Unitary Operation

YUECHEN LI, School of Automation Science and Electrical Engineering, Beihang University, Beijing, China

HANYU PEI, School of Automation Science and Electrical Engineering, Beihang University, Beijing, China

LINZHI HUANG, School of Automation Science and Electrical Engineering, Beihang University, Beijing, China

BEIBEI YIN, School of Automation Science and Electrical Engineering, Beihang University, Beijing, China

KAI-YUAN CAI, School of Automation Science and Electrical Engineering, Beihang University, Beijing, China

With the continuous advancement of quantum computing (QC), the demand for high-quality quantum programs (QPs) is growing. To avoid program failure, in software engineering, the technology of automatic program repair (APR) employs appropriate patches to remove potential bugs without the intervention of a human. However, the method tailored for repairing defective QPs is still absent. This article proposes, to the best of our knowledge, a new APR method named UniTAR that can repair QPs via unitary operation automatically. Based on the characteristics of superposition and entanglement in QC, the article constructs an algebraic model and adopts a generate-and-validate approach for the repair procedure. Furthermore, the article presents two schemes that can respectively promote the efficiency of generating patches and guarantee the effectiveness of applying patches. For the purpose of evaluating the proposed method, the article selects 29 mutated versions as well as five real-world buggy programs as the objects and introduces two traditional APR approaches GenProg and TBar as baselines. According to the experiments, UniTAR can fix 23 buggy programs, and this method demonstrates the highest efficiency and effectiveness among three APR approaches. Besides, the experimental results further manifest the crucial roles of two constituents involved in the framework of UniTAR.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Computer systems organization** → **Quantum computing**;

Additional Key Words and Phrases: Quantum computing, automatic program repair, quantum software engineering, unitary operation, software cybernetics, S-ADA

Authors' addresses: Yuechen Li, Beihang University, School of Automation Science and Electrical Engineering, Beijing, Beijing, China; e-mail: liyuechen@buaa.edu.cn; Hanyu Pei, Beihang University, School of Automation Science and Electrical Engineering, Beijing, Beijing, China; e-mail: peihanyu@buaa.edu.cn; Linzhi Huang, Beihang University, School of Automation Science and Electrical Engineering, Beijing, Beijing, China; e-mail: huanglinzhi@buaa.edu.cn; Beibei Yin (Corresponding author), Beihang University, School of Automation Science and Electrical Engineering, Beijing, Beijing, China; e-mail: yinbeibei@buaa.edu.cn; Kai-Yuan Cai, Beihang University, School of Automation Science and Electrical Engineering, Beijing, Beijing, China; e-mail: kycai@buaa.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1049-331X/2024/06-ART154

<https://doi.org/10.1145/3664604>

ACM Reference Format:

Yuechen Li, Hanyu Pei, Linzhi Huang, Beibei Yin, and Kai-Yuan Cai. 2024. Automatic Repair of Quantum Programs via Unitary Operation. *ACM Trans. Softw. Eng. Methodol.* 33, 6, Article 154 (June 2024), 43 pages. <https://doi.org/10.1145/3664604>

1 INTRODUCTION

As an emerging and distinctive form of computing, **quantum computing (QC)** [15] has the potential to solve extremely complex computational problems and implement specific simulations credited to the merit of quantum parallelism. Unlike **classical computing (CC)** operating upon bits, QC works with qubits as its fundamental unit. A qubit is utilized and manipulated to perform a particular operation through a series of quantum gates. With these gates, **quantum programs (QPs)** can be designed to execute certain algorithms and realize particular functionality in virtual or real quantum systems. Nowadays, several notable programming languages are available for quantum programming, such as IBM's Qiskit [2],¹ Google's Cirq, Microsoft's Q#, and Origin Quantum's QPanda.

As QC gets increasingly applied, it becomes more vital to ensure the correctness and reliability of QPs. Analogous to **classical programs (CPs)**, QPs also suffer from defects that may trigger program failures. From the perspective of software engineering, to make software an autonomous, dependable, and affordable system [6], the approaches of **automatic program repair (APR)** could be adopted to eliminate software defects and then enhance the quality of the whole software system. Automatically introducing appropriate patches contributes to reducing the cost incurred by human involvement, such as patch placement and effectiveness verification. The principal steps of repairing CPs, like localization, patch generation, and patch validation [20, 57], have been studied in previous research. Explicitly, software testing examines the behavior of the software under test and provides bug reports with detailed information on software defects. Based on the bug reports, the localization technique assists in determining where to add patches. Then the patches are iteratively generated by searching in specific spaces and validated by test cases until defects are removed.

In contrast to CPs, quantum measurement is the only means of acquiring quantum information in a real quantum system. This operation inevitably results in qubit collapse, i.e., the quantum state is irreversibly converted to a classical one. Measurement may challenge the observation and inspection of inner states after executing certain codes, such as localizing faults in repairing CPs. Hence, the direct application of traditional APR approaches to repair QPs appears impractical and ineffective. Actually, the nature of complying with superposition and entanglement in QC allows qubits to store more latent information and conduct reversible operations before measurement. Therefore, there is a clue that an innovative and complete method of repairing QPs can be proposed to make full use of the distinctive characteristics of QC.

In the related works of QPs, there are three main types of bug patterns summarized by Luo et al. [38], including **application programming interface- (API)** related defects, incorrect application logic, and math-related defects. The first two kinds are inclined to cause the program to crash; that is, the program terminates accidentally while running. By contrast, the last one produces incorrect functionality without inducing program crash. Intuitively, developers should pay more attention to math-related defects in particular scenes, as they cannot be identified until the program is sufficiently analyzed and evaluated by test cases instead of affirming through the occurrence of a program crash. The math-related defects stressed in the article result from the

¹<https://qiskit.org>

unexpected application of quantum gates when developers make mistakes in program development. According to quantum mechanics, a quantum gate is denoted as the concrete manipulation of qubit(s), where a stringent requirement for unitary operation lies, the operation that belongs to a special kind of linear transformation. Then, such defects frequently trigger prominent divergence from the expected unitary operation inside a QP, i.e., in vector space, the actual final state just before the measurement is inconsistent with the required one. In the view of algebra, introducing a unitary operator as a patch can possibly prevent the undesirable impact caused by defects. In total, multiple defects ultimately map the correct result to the wrong one, while the linear patch can conduct an inverse operation to remove defects.

This article proposes UnitAR, an APR approach for QPs via unitary operation. Specifically, the QP before being repaired with additional patches can be abstracted as a black-box model, where any accessible information merely depends on the inputs and outputs. Without modifying the original codes used to implement the qubit evolution, the patch is just located before or after the process of evolution. With adequate information about initial (quantum) states and measurement results through testing, adopting suitable estimation for the black box aids in constructing the search space to generate candidate patches. If a patched version is validated to pass all the tests, then it can be seen as a fixed program. Moreover, since any unitary operation to qubits can be abstracted as one operator and the repair procedure is implemented in an algebraic manner, the proposed method enables the removal of multiple defects fully automatically.

Apart from the elementary procedure, two key points to refine the method and upgrade the feasibility are included as well. On the one hand, mathematical constraints are proposed to speed up patch searching and decrease the complexity of patch validation. On the other hand, repeated measurements and floating-point errors bring disturbance to the generated patches that should be unitary operators in theory. To cope with that, an algebraic procedure via matrix decomposition can guarantee the unitarity of patches within a certain precision. Then, the article conducts empirical studies to evaluate UnitAR, where 34 object programs (29 mutated versions and 5 real-world buggy programs) and two baseline APR approaches (GenProg and TBar) are included. The results demonstrate that compared to two baselines, UnitAR generally has higher fix rates and consumes less time. The two proposed propositions can simplify candidate patches and reduce execution time in most cases. The procedure of unitary correction enables the generated patch to approximate a strictly unitary operator with machine precision.

Recent works have researched some issues related to APR for QC, such as locating possible buggy segments divided from a complete QP [49] and applying the large language model to generate patches with human intervention [21]. Nevertheless, the entire procedure to repair QPs in a fully automatic way seems still absent. To the best of the authors' knowledge, the work described and evaluated in the article is possibly the first one that specifically targets the APR issue of QPs and presents an automatic repair procedure without human intervention. The main contributions of this article are concisely listed as follows:

- Based on the algebraic operation and the black-box model, the article forms a complete automatic repair approach for QPs in theory, where the proposed approach UnitAR can be implemented by the classical algorithm.
- For the generate-and-validate approach, the article proposes and adopts some particular algebraic propositions to optimize the efficiency of the proposed method.
- The article includes an effective procedure that can convert non-unitary patches to unitary ones via matrix decomposition. The procedure can guarantee the success of patching during program repair.

The rest of the article is organized as follows. Section 2 discusses the previous works relevant to the testing of quantum software and the automatic repair of classical programs. Section 3 presents the preliminaries of quantum computing and quantum information more specifically. In Section 4, the procedure of program repair is theoretically modeled and analyzed. Then, empirical studies are included in Section 5 to evaluate the proposed method. Finally, the article draws a conclusion and outlines potential directions for future research in Section 6.

2 RELATED WORK

This section contains the related work from two aspects: one relevant to studies on the testing of quantum programs and the other about the traditional technologies of automatic program repair widely applied in CPs.

2.1 Testing of Quantum Software

The testing technology applied for quantum software comprises testing general software using quantum algorithms and testing quantum software via classical algorithms, where researchers currently focus on the latter. With the increasing attention to testing quantum software [19, 42], plenty of studies have been conducted recently and explored to migrate the techniques from CC to QC. The principal way is to apply classical algorithms to testing and analyzing QPs. Ali et al. [3] have defined basic concepts of testing QPs such as test suite and test oracle and then proposed Quito, including three kinds of coverage criteria upon inputs and outputs. They have employed mutation analysis to generate defective versions of QPs to evaluate Quito's effectiveness. In software engineering, mutation testing (or mutation analysis) involves modifying a program in small ways to evaluate the existing tests. Hence, several studies [17, 18, 41, 56] have investigated its application for QPs. Mendiluze et al. [41] have introduced Muskit, where a set of mutation operators has been adopted to generate mutated versions systematically. Fortunato et al. [16, 18] have proposed QMutPy through a novel set of mutation operators based on quantum measurements and quantum gates. As a prevalent Python-based toolset, QMutPy is able to perform mutation testing written in Qiskit's full-stack library automatically. Furthermore, to discover the defects bugs inside QPs through the testing phase, several prevailing testing techniques used in CPs have been attempted, such as property-based testing [23], fuzz testing [51], differential testing [52], combinatorial testing [53], search-based testing [54], and so on.

In the event of output failure identified by the test oracle, researchers tend to seek out what kinds of defects trigger the unexpected consequence. Considering the limitation to probing and interpreting inner states of QPs, Huang et al. [26] have presented quantum program assertions based on statistical tests to judge if a state matches the expected value. Additionally, they have shared some types of possible bugs, such as incorrect quantum initial values, incorrect operations and transformations, and incorrect composition of operations using iteration or recursion. In response to specific bugs, Zhao et al. [63] have studied identifying and categorizing bug patterns upon Qiskit. Due to the lack of bug patches for QPs, Luo et al. [38] have investigated real-world bugs along with their fixes, which are collected from the prevalent quantum programming languages. The study has shown quantum-specific bugs in a high proportion and pointed out the crucial type—math-related bugs, where wrong numerical computation and randomness handling are responsible for those.

Existing studies have started preliminary steps in testing quantum programs. However, for the subsequent process after testing, there still lies the intensive demand for a method of repairing QPs to improve the quality of programs. As a solution, a QC-oriented method that matches the nature of QC and takes advantage of potential information should be proposed to improve QPs effectively and efficiently.

2.2 Automatic Program Repair

After evaluating the defective programs through testing, further steps are required to eliminate the latent defects. Toward CPs, *program repair techniques* aim to automatically repair software systems by producing an actual patch until they are finally accepted or adapted to fit the system properly [20]. The patch is a permanent solution to a software bug [47], which always alters the program code and is expected to implement solutions of the same quality as the one produced by developers. Based on test cases and test oracle, the repairing process for CPs comprises three steps: localization, patch generation, and patch verification [57].

For the part of localization, **spectrum-based fault localization (SBFL)** has been proposed as a general mechanism to localize the possible faulty statement [48]. *Suspiciousness* has been applied to determine the opting probability of the program entity during automatic program repair. Some studies have exploited SBFL and extended it to four types of suspiciousness, i.e., Tarantula [28], Ochiai [1], GenProg [30, 31], and Jaccard [9]. Compared to fault localization with a focus on faulty statements, *fix localization* identifies the program locations suitable for the synthesis of fixes, regardless of the location of the faults. In light of that, some technologies like *model-based fix localization* [11, 45, 46, 58] and *angelic fix localization* [12, 40] have been widely investigated and researched.

For the generate-and-validate approaches, the search-based algorithm, as one of the principal strategies, employs operators randomly or guided by a heuristic or meta-heuristic algorithm to search for appropriate alternative patches. As a pioneering study, Weimer et al. [60] have proposed GenProg that utilizes *genetic programming* to generate patches. This method applies operations analogous to natural genetic processes to the population of programs. Whether the patch plays a role in repairing is scarcely known until each possible patch is verified after its addition to the defective version. Nevertheless, checking enormous patches may lead to extremely high costs. To optimize the process by reducing the alternative patches to be validated, Weimer et al. [59] have proposed *adaptive equivalence* based on semantic similarity. After that, Long et al. have put forward the method of *staged program repair* [34] and Prophet [35] by sorting patches and prioritizing ones with higher scores.

With the development of benchmarks for APR (such as Defects4J [29] for Java programs), several studies like SimFix [27], TBar [32], and Recorder [64] have utilized the previous data or patterns to assist in repairing the current programs. SimFix makes use of complementary information from both existing patches in the existing projects and similar code snippets in the faulty project. As a template-based repair method, TBar applied the refined set of fix patterns collected and summarized from previous literature data to repair the buggy programs in an effective manner. Recorder presents a syntax-guided edit decoder based on deep learning, which can outperform the above-mentioned two traditional APR approaches. Thus, data-driven approaches have increasingly become mainstream in repairing CPs.

However, owing to the characteristic of superposition and entanglement in QC, directly applying the traditional APR methods proposed for CPs to repairing QPs may have two limitations. On the one hand, almost every qubit employed in a QP is manipulated and processed, so test cases in the form of quantum states are likely to reach the same coverage of code paths, which poses a challenge for fault localization. Sato et al. have proposed to divide the QP into several segments and then locate the buggy part by testing each segment in a reasonable way [49]. The execution of numerous segments and the requirement of additional test oracle for each segment indicate higher consumption to locate bugs in QPs compared to applying the existing fault localization methods for CPs. On the other hand, any qubit can interact with each other through controlled quantum gates, such that an introduced patch on one qubit may lead to the wrong operation of other qubits.

To handle that, the method proposed in the article simply locates patches at the code lines before or after the part of qubit evolution, because it is convenient to analyze related quantum states for the purpose of generating suitable patches. Besides, the article represents patches in the form of unitary operators that can process multiple qubits in one breath.

3 PRELIMINARIES

3.1 Quantum State

For a classical bit, its only state is either 0 or 1. By contrast, the possible states of a qubit are other than $|0\rangle$ or $|1\rangle$. The notations *ket* $|\cdot\rangle$ and *bra* $\langle\cdot|$ are called Dirac notations in quantum mechanics, where ket refers to the *Hermitian transpose* of bra, written by conjugating $*$ and transposing $\top$ or one notation *dagger* $\dagger$ that $|\cdot\rangle = \langle\cdot|^\dagger = (\langle\cdot|^*)^\top$. The state $|\varphi\rangle$ of a single qubit is calculated in *Hilbert space* $\mathcal{H}$, a kind of vector space equipped with an inner product and certain mathematical properties like completeness.² Owing to vector space, the state $|\varphi\rangle$ can be formed by linear combination called *superposition*,

$$|\varphi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad (1)$$

where $\alpha, \beta \in \mathbb{C}$ are *amplitudes* of this state, and the basic states $|0\rangle$ and $|1\rangle$ known as *computational basis states* can be written as standard bases that $|0\rangle = [1, 0]^\top$, $|1\rangle = [0, 1]^\top$. After measuring the state $|\varphi\rangle$, the result is either 0 with probability $|\alpha|^2$ or 1 with probability $|\beta|^2$. For the *normalization condition that probabilities must sum to 1*, it is natural that the equation $|\alpha|^2 + |\beta|^2 = 1$ holds or the inner product meets $\langle\varphi|\varphi\rangle = 1$.

Considering a composite system made of multiple qubits, its state space can be expressed as the *tensor product* (or *Kronecker product*) of component qubits via the notation $\otimes$. If the qubits $|\varphi_i\rangle$ numbered from 0 to $n-1$ are prepared, then their joint state $|\varphi\rangle$ is written as

$$|\varphi\rangle = |\varphi_{n-1}\rangle \otimes |\varphi_{n-2}\rangle \otimes \cdots \otimes |\varphi_0\rangle. \quad (2)$$

Example 1 shows the formation of a concrete joint quantum state.

Example 1. Given the following two quantum states:

$$|\varphi_0\rangle = \frac{\sqrt{3}}{3}|0\rangle + \left(\frac{\sqrt{6}}{6} - \frac{\sqrt{2}}{2}i\right)|1\rangle, \quad |\varphi_1\rangle = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i\right)|0\rangle + \left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}i\right)|1\rangle,$$

their joint state $|\varphi\rangle \equiv |\varphi_1\varphi_0\rangle \equiv |\varphi_1\rangle|\varphi_0\rangle \equiv |\varphi_1\rangle \otimes |\varphi_0\rangle$ is shown as

$$|\varphi\rangle = \begin{bmatrix} \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i \\ \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}i \end{bmatrix} \otimes \begin{bmatrix} \frac{\sqrt{3}}{3} \\ \frac{\sqrt{6}}{6} - \frac{\sqrt{2}}{2}i \end{bmatrix} = \begin{bmatrix} \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i\right) \cdot \begin{bmatrix} \frac{\sqrt{3}}{3} \\ \frac{\sqrt{6}}{6} - \frac{\sqrt{2}}{2}i \end{bmatrix} \\ \left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}i\right) \cdot \begin{bmatrix} \frac{\sqrt{3}}{3} \\ \frac{\sqrt{6}}{6} - \frac{\sqrt{2}}{2}i \end{bmatrix} \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{6}}{6} + \frac{\sqrt{6}}{6}i \\ \frac{3+\sqrt{3}}{6} + \frac{-3+\sqrt{3}}{6}i \\ \frac{\sqrt{6}}{6} - \frac{\sqrt{6}}{6}i \\ \frac{-3+\sqrt{3}}{6} - \frac{3+\sqrt{3}}{6}i \end{bmatrix}$$

or

$$|\varphi\rangle = \left(\frac{\sqrt{6}}{6} + \frac{\sqrt{6}}{6}i\right)|0\rangle_2 + \left(\frac{3+\sqrt{3}}{6} + \frac{-3+\sqrt{3}}{6}i\right)|1\rangle_2 + \left(\frac{\sqrt{6}}{6} - \frac{\sqrt{6}}{6}i\right)|2\rangle_2 + \left(\frac{-3+\sqrt{3}}{6} - \frac{3+\sqrt{3}}{6}i\right)|3\rangle_2.$$

In this article, a Dirac notation $|x\rangle_n$ with a subscript n underscores x as a decimal value and the state composed of n qubits. In contrast, the notation $|x\rangle$ without a subscript indicates x as a binary value. For instance, $|1\rangle_2 = |01\rangle$ and $|1\rangle_3 = |001\rangle$.

²Completeness of a vector space has a mathematical definition that every Cauchy sequence of points in a vector space V has a limit that is also in V .

```

1  qbit1 = QuantumRegister(1, name='qbit1')    # define the first qubit qbit1
2  qbit2 = QuantumRegister(1, name='qbit2')    # define the second qubit qbit2
3  cbit1 = ClassicalRegister(1, name='cbit1')   # define the first bit cbit1
4  cbit2 = ClassicalRegister(1, name='cbit2')   # define the second bit cbit2
5  qc = QuantumCircuit(qbit1, qbit2, cbit1, cbit2) # construct the quantum circuit
6  qc.h(qbit1)                                # superposition of 0 and 1
7  qc.cx(qbit1, qbit2)                        # entangle two qubits
8  qc.measure(qbit1, cbit1)                   # measure qbit1 and store the result into cbit1
9  qc.measure(qbit2, cbit2)                   # measure qbit2 and store the result into cbit2

```

Fig. 1. Example: The Python Program EQ (Entangled Qubits) upon Qiskit.

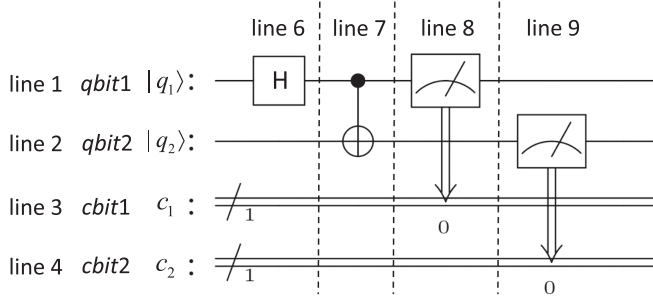


Fig. 2. Example: The quantum circuit of the above quantum program.

3.2 Quantum Circuit

Apart from basic definitions of variables, the phases of qubit *evolution* and qubit *measurement* play a chief role in manipulating qubits to realize the functionality of a QP, where the former phase refers to qubit operation with time, and the latter phase stands for observation of quantum states. The entire quantum program can be pictured as a corresponding quantum circuit, and their connection is clarified in Example 2.

Example 2. The QP named EQ and its quantum circuit with two qubits entangled are shown in Figure 1 and Figure 2. The program entangles *qbit1* with *qbit2* and then stores the observed results in bits *cbit1* and *cbit2*, respectively. When measuring the final state, the results of two qubits will always remain the same. The Python program is written upon the quantum language Qiskit, including defining qubits, constructing the quantum circuit, performing operations, and measuring the final states. Quantum gates (lines 6 to 7) are applied to entangle two qubits such as $|0\rangle|0\rangle \mapsto (|0\rangle|0\rangle + |1\rangle|1\rangle)/\sqrt{2}$, and quantum measurements (lines 8 to 9) are responsible for obtaining the qubit state stored in two bits like $(|0\rangle|0\rangle + |1\rangle|1\rangle)/\sqrt{2} \mapsto (00)_b = 0$ or $(|0\rangle|0\rangle + |1\rangle|1\rangle)/\sqrt{2} \mapsto (11)_b = 3$ with the even probability distribution.

3.2.1 Quantum Gate. Unlike logic gates in CC, quantum gates carry around and process the quantum information in Hilbert space. For any quantum gate U , it performs a linear map within Hilbert space that $U \in \mathcal{L}(\mathcal{H}, \mathcal{H})$, where $\mathcal{L}(\cdot, \cdot)$ denotes all the linear operators from the left to the right one. In terms of finite dimensions $\dim(\mathcal{H}) = 2^n$ with n qubit(s), the quantum gate can even be represented by a specific $2^n \times 2^n$ complex matrix, $U = [|u^{(1)}\rangle, |u^{(2)}\rangle, \dots, |u^{(2^n)}\rangle]$. Noticeably, the only constraint is to guarantee U a *unitary operator*, because it retains the normalization condition that $\langle \varphi | U^\dagger U | \varphi \rangle = \langle \varphi | \varphi \rangle = 1$. In detail, U is a unitary operator iff $U^\dagger U = I$ or $\langle u^{(m)} | u^{(m')} \rangle = \delta_{m,m'} (m, m' = 1, 2, \dots, 2^n)$, where $\delta_{m,m'}$ is generally known as *Dirac delta function* that $\delta_{m,m'} = 1$ only if $m = m'$, and otherwise $\delta_{m,m'} = 0$.

According to the number of processed qubits n , quantum gates can be classified into single qubit gates ($n = 1$) and multiple qubit ($n \geq 2$) gates. Familiar single qubit gates include Pauli matrices σ_x (x), σ_y (y), σ_z (z), Hadamard gate H (h), rotation operators with parameter θ like $R_x(\theta)$ (rx), $R_y(\theta)$ (ry), $R_z(\theta)$ (rz), and so on. As a typical type of multiple qubit gates, *controlled- U* gates (such as cx , cy , cz) implement the structure analogous to if-then, where the processed qubits are divided into the control ones (if) and target ones (then).

3.2.2 Quantum Measurement. Measurement is an essential step in observing the closed quantum system and then seeking out what is going on inside it. Any interaction with the outer environment will make the system not confined to unitary evolution. On a real quantum computer, measurement takes an indispensable role in verifying whether the program runs as expected.

According to the postulate in quantum mechanics, quantum measurements are described by a collection of projectors $\{M_y\}_{y=0}^{2^n-1}$, where the index y refers to the measurement result that may occur in the outcomes from an n -qubit QP. Without loss of generality, the projectors operating on state $|\varphi\rangle$ can be assigned in computational basis states. Supposing the projector M_y is embodied as $M_y = |y\rangle\langle y|$ and the collection $\{|y\rangle\}_{y=0}^{2^n-1}$ consists of 2^n computation basis states, the probability of the random variable Y satisfying $Y = y$ can be understood as the inner product of the state after M_y acting on $|\varphi\rangle$, such that

$$p_Y(y) = |\langle\varphi|y\rangle|^2 = \langle\varphi|M_y^\dagger M_y|\varphi\rangle = \langle\varphi|M_y|\varphi\rangle. \quad (3)$$

Once $Y = y$ occurs, the state will immediately convert to $M_y|\varphi\rangle/\sqrt{p_Y(y)}$, which is called the *collapse* of quantum states. In view of quantum measurement as the inner product, the conclusion of measuring $|\varphi\rangle$ in Section 3.1 can be verified in Example 3.

Example 3. Refer to Equation (3) and given that $P_0 = |0\rangle\langle 0|$, $P_1 = |1\rangle\langle 1|$, $|\varphi\rangle = [\alpha, \beta]^T$, it is evident that $p_Y(0) = |\langle\varphi|0\rangle|^2 = |\alpha|^2$, $p_Y(1) = |\langle\varphi|1\rangle|^2 = |\beta|^2$. After that, the state promptly collapses to either $\alpha|0\rangle/|\alpha|$ or $\beta|1\rangle/|\beta|$, corresponding to two cases of measurement.

3.3 Black-box Model of Quantum Programs

Any transformation through the quantum gate can be algebraically denoted as a singular unitary operation. Let $|\varphi_i\rangle (i = 1, 2, \dots, t)$ be the joint state after the i th gate operation, in which the initial and final state are respectively defined as $|\varphi\rangle \equiv |\varphi_0\rangle$ and $|\psi\rangle \equiv |\varphi_t\rangle$. On condition that there are t times operations U_i acting on all the qubits in an n -qubit QP, and then for each operation, the equation $|\varphi_i\rangle = U_i|\varphi_{i-1}\rangle$ holds. Considering the QP abstracted as a black-box model, the entire evolution is substituted by U , merely linking the input with the output. Thus,

$$|\psi\rangle = U_t |\varphi_{t-1}\rangle = U_t U_{t-1} |\varphi_{t-2}\rangle = U_t U_{t-1} \dots U_1 |\varphi\rangle = U |\varphi\rangle,$$

where $U = U_t U_{t-1} \dots U_1$ and it is obvious to identify U as a unitary operator by $U^\dagger U = I$. Example 4 proposes an approach to denote the evolution of qubits as one unitary operator U .

Example 4. To analyze the QP shown in Figure 2, the initial state $|\varphi\rangle$ is set to $|q_2\rangle|q_1\rangle$, and the operator U is

$$U = U_{CN}(\mathbb{1}_{2 \times 2} \otimes H),$$

where the identity $\mathbb{1}_{2 \times 2}$ stands for no operation for the high qubit.

In fact, there exists a limitation to determining U in the above way. On the one hand, the complexity of calculating matrices will increase significantly in the case of large-scale quantum gates. On the other hand, the inherent process of a QP cannot be known except by measurement. Consequently, merely based on the initial and final states, U should be calculated or estimated in another feasible manner, the method that will be explained in detail in Section 4.4.

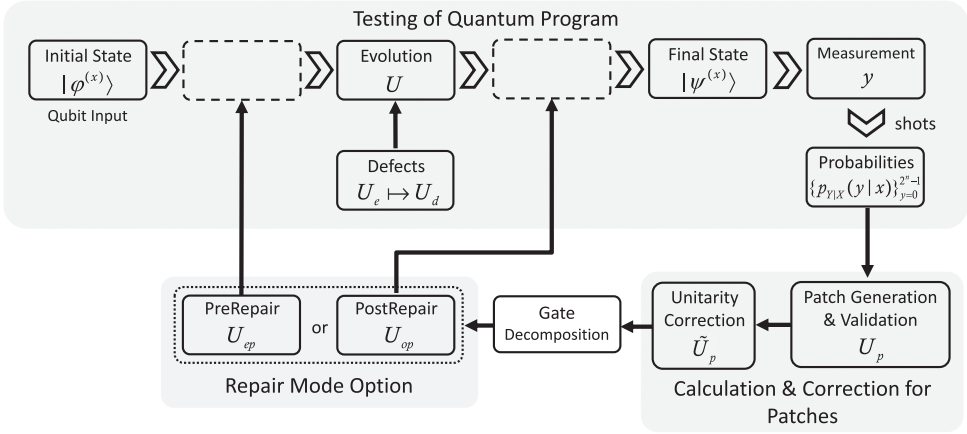


Fig. 3. The framework of UnitAR. The procedure is mainly composed of the testing phase (i.e., “Testing of Quantum Program”) and repairing phase (i.e., “Calculation & Correction for Patches” and “Repair Mode Option”).

4 METHODOLOGY

4.1 Overview of the Method

The framework of the proposed method UnitAR is shown in Figure 3. UnitAR consists of two main parts, i.e., the testing phase and the repairing phase. At first, in light of the black-box model, adequate test cases are input to the QP as initial states. After qubits are processed by qubit evolution and quantum measurement, it is accessible to acquire bit outputs, especially for the approximate probability distribution through stochastic sampling.

Then, according to the test results, plausible patches can be produced through the procedure of patch generation and validation. Once any of the patches is identified, it is further processed by unitary correction, where introducing this process promotes the patch to be a legitimate unitary operator, thereby enabling the corrected one to be decomposed into basic quantum gates. Eventually, based on the repair mode that is randomly selected, patches are added to the program either before or after the code lines of evolution. Practically, three key parts, including modeling of the repair procedure, patch generation and validation, and unitary correction, are the determinants of the success of program repair, which will be discussed by order from Section 4.3 to Section 4.5.

Under certain circumstances, it is necessary to decompose the unitary patch into basic gates for the purpose of better readability and maintainability. However, the specific method to implement gate decomposition is not considered as the core in this article. A universality result is true for quantum computation, where a set of gates is said to be *universal for quantum computation* if any unitary operation may be approximated to arbitrary accuracy by a quantum circuit involving only those gates, for example, that Hadamard, phase, and $\pi/8$ gates form a family of such gates [43]. In other words, the patch is denoted as a series of quantum gates performing the same linear operation and can be successfully added to the defective programs. Nevertheless, the specific techniques of decomposition have been mature enough, and the available API like `transpile()` in Qiskit can be directly used to attain this functionality.

4.2 Assumptions

Based on the characteristics of QP, the following assumptions are made for the proposed approach:

- The defects prompting the program to output failures merely occur during the qubit evolution.
- 2^n test suites for the n -qubit QP can be obtained and the corresponding test cases form a collection of orthonormal basis.
- For each version under test, all the qubits will be measured.

There are explanations for the proposed assumptions by order.

- The symptom of defects, namely *incorrect functionality*, has been summarized by Chen et al. [8] in understanding bugs of the deep learning procedure. It refers to the objective behaving incorrectly but does not crash, such as producing unexpected results, undesirable model structures, or incorrect intermediate states, which can also be used to explain phenomena in QPs. Qubit evolution as the core for QPs, dominates qubit operating and accomplishes specific functions. Even though executing the program with no crash, the defects hidden in the evolution phase will seriously affect the program's functionality, thereby triggering output failures. In such a situation, how to erase the symptom incorrect functionality and remove the defects during the qubit evolution are emphasized in the article.
- The input domain is determined by particular requirements or program specifications. In fact, in terms of identifying whether a patched version is fixed, sufficient test cases are usually required, because executing only a few test cases may take an unfixed version as a fixed one for mistake. In the view of Hilbert space corresponding to an n -qubit QP, 2^n orthonormal bases as the initial states can represent arbitrary quantum states by linear combination and span into the 2^n -dimensional linear space. Besides, 2^n test suites that include the above orthonormal bases along with their expected output states can contribute to determining the unitary operation of the QP, where more details are explained in Proposition 1.
- In the view of *principle of implicit measurement*, without loss of generality, any unterminated quantum wires (qubits that are not measured) at the end of a quantum circuit may be assumed to be measured [43]. If any qubit remains unmeasured, then it is impossible to determine the invertible operation due to the loss of information. As a result, the original QPs to be repaired are either ones where all the qubits are measured or those where it is possible to add measurement operations to unmeasured qubits without altering the program functionality.

4.3 Primary Modeling of the Repair Procedure

For the purpose of automation, the repair procedure can be implemented in an algebraic manner. Its elementary modeling involves how the testing phase is conducted, how to utilize the test results for repair, and how patches are determined.

4.3.1 Testing Phase. In the field of software testing, coverage-based testing is a common means to evaluate the software under test thoroughly. Different from n -bit CPs with the 2^n test cases at most, there are more possibilities for test cases in QPs. They can be selected from any state vector $|\varphi\rangle$ in Hilbert space $\mathcal{H}$ with n qubit, in spite of the test set only composed of finite test cases, where $\mathcal{H}$ and $|\varphi\rangle$ are required that $\dim(\mathcal{H}) = 2^n$ and $\langle\varphi|\varphi\rangle = 1$.

As for the input-coverage testing, it is necessary to cover all the vital test cases in the particular domain as possible. Let TC_E and TC_V be the test sets with the test cases designed for evaluating the defective program version and validating the repaired program version respectively, where it is worth noting that both TC_E and TC_V are finite sets rather than vector space, and to be simplified, their sizes are noted that $k_E = |TC_E| < \infty$ and $k_V = |TC_V| < \infty$. Hence, the chosen test cases should be representative enough, and for example, the collective of computational basis states $\{|x\rangle\}_{x=0}^{k_E-1}$ can construct a reasonable TC_E for an n -qubit QP.

Let the expected, defective and patched versions be respectively denoted as QP_E , QP_D , and QP_P , where the expected program version is the direction of what the defective version is repaired toward. In other words, program repair aims to rectify QP_D into QP_P with proximity to QP_E as possible. Additionally, QP_E can be abstracted as an ideal and virtual program with all the defects removed according to program specifications.

From the perspective of the black-box model, evolution in QP_E and QP_D can be separately indicated as the unitary operator U_e and U_d . Let the initial state be $|\varphi^{(x)}\rangle \in \mathcal{H}_i$ and the final states of the two versions respectively be $|\psi_e^{(x)}\rangle \in \mathcal{H}_e$ and $|\psi_d^{(x)}\rangle \in \mathcal{H}_d$, where the test input of the test case $test_x$ is defined as the initial state $|\varphi^{(x)}\rangle$. With regard to $\mathcal{H}_i$, $\mathcal{H}_e$, and $\mathcal{H}_d$, they refer to each subspace in Hilbert space $\mathcal{H}$ corresponding to three types of versions. Then, two equations can be formed as

$$\begin{aligned} |\psi_e^{(x)}\rangle &= U_e |\varphi^{(x)}\rangle, \\ |\psi_d^{(x)}\rangle &= U_d |\varphi^{(x)}\rangle. \end{aligned} \quad (4)$$

According to the input and output for the evolution phase, the test suite TS_E and TS_V for evaluation and validation are defined as

$$\begin{aligned} TS_E &\equiv \left\{ \left(|\varphi^{(x)}\rangle, |\psi_e^{(x)}\rangle \right) \middle| |\varphi^{(x)}\rangle \in TC_E \right\}, \\ TS_V &\equiv \left\{ \left(|\varphi^{(x)}\rangle, |\psi_e^{(x)}\rangle \right) \middle| |\varphi^{(x)}\rangle \in TC_V \right\}. \end{aligned} \quad (5)$$

Practically, TS_E and TS_V can be generated according to the corresponding formula-based program specification [36]. In other words, under certain circumstances, the expected behavior of a QP can be represented by a formula $|\varphi\rangle \mapsto |\psi\rangle$. For some programs to implement particular algorithms such as **Quantum Fourier Transform (QFT)**, the available software development kits (e.g., Qiskit) widely apply the formula $|j\rangle \mapsto \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i j k / 2^n} |k\rangle$ ($j, k \in \mathbb{N}$) to depict the API. For some of the other programs, the full program specification can be determined according to the expected operation of quantum gates. Taking the EQ (shown in Example 2) for an instance, a Pauli-X gate acts on the first qubit, and then a CNOT gate operates on both two qubits, so the formula can be formed as $|\varphi_1\rangle|\varphi_0\rangle \mapsto U_{CN}(|\varphi_1\rangle \otimes H|\varphi_0\rangle)$. Based on the corresponding formula, it is quite straightforward to generate the test suites in the form of classical vectors.

In the testing phase, test cases are usually chosen following a particular probability distribution, like the prevailing testing strategies *random testing* [13, 22], *adaptive random testing* [10, 25], *random partition testing* [5], and *dynamic random testing* [39, 44, 50]. The property of probability wave implies the randomness of quantum states, in either pure or mixed states. A random variable, denoted by X , is used to indicate that test case has been chosen as the input for the program. If $X = x$, then this signifies that $|\varphi^{(x)}\rangle$ is selected from TC_E or TC_V . Then, it is clear that the result variable Y depends on the test case variable X , so the test results can be comprehended as the distribution of conditional probability with the collection of projectors $\{M_y\}_{y=0}^{2^n-1}$, such that

$$p_{Y|X}(y|x) = \langle \varphi^{(x)} | U^\dagger M_y U | \varphi^{(x)} \rangle. \quad (6)$$

Then, from the perspective of automatic patch validation, a *fixed* patched version QP_R that outputs the result Y_R and the expected one QP_E along with the result Y_E are linked with the relation that

$$\forall x \in \{0, 1, \dots, k_V - 1\}, \forall y \in \{0, 1, \dots, 2^n - 1\} : p_{Y_R|X}(y|x) = p_{Y_E|X}(y|x).$$

According to *the law of large numbers*, the frequency of $Y = y$ can approach the probability distribution in the case of enough shots. In fact, arbitrary testing results cannot be strictly identical to the expected distribution, even though the program is repaired successfully in intuition. To define the test oracle determining whether a test has passed or failed, the article adopts **Jensen–Shannon**

divergence (J-S divergence), i.e., a kind of classical statistical distance, to quantify the similarity between two probability distributions. For random variables Y_1 and Y_2 along with corresponding probability distributions $\{p_{Y_1}(y)\}_{y=0}^{2^n-1}$ and $\{p_{Y_2}(y)\}_{y=0}^{2^n-1}$, their J-S divergence is expressed as

$$\text{JSD}(Y_1||Y_2) \equiv \frac{1}{2} \sum_{y=0}^{2^n-1} \left(p_{Y_1}(y) \log \frac{p_{Y_2}(y)}{p_{Y_1}(y)} + p_{Y_2}(y) \log \frac{p_{Y_1}(y)}{p_{Y_2}(y)} \right), \quad (7)$$

where it is regulated that $0 \log 0 = 0$, and the natural logarithm is usually chosen. For J-S divergence, the smaller value represents the closer distance between two probability distributions. According to the required precision, a configurable threshold v_T ($v_T > 0$) can be set. Then the guideline of test oracle is formulated that $\text{JSD}(Y_E||Y_R|X) < v_T$ means the test case $test_x$ passes and otherwise fails. Actually, an appropriate setting of v_T may be impacted by the number of shots and qubits. Note that regardless of the randomness related to shots and qubits, two outcomes $Y_D^1|X$ and $Y_D^2|X$, deriving from executing the defective QP_D with the same test case $test_x$ twice, must satisfy $\text{JSD}(Y_D^1||Y_D^2|X) < v_T$. By this idea, the article introduces a setting of v_T that can vary with different QPs, i.e.,

$$v_T = \max_X \{ \text{JSD}(Y_D^1||Y_D^2|X), \varepsilon \}, \quad (8)$$

where ε indicates the minimum tolerant error. Under certain circumstances, $\max_X \text{JSD}(Y_D^1||Y_D^2|X)$ may be calculated as 0. Therefore, a small value ε , such as 0.005, is introduced to prevent $v_T = 0$, or even a fixed version may fail all the tests. Example 5 shows how J-S divergence works as the test oracle to identify the test result.

Example 5. Let the outcomes of a patched version be $\{(y|x, n_y)\}$, where n_y represents the times that the result $Y_R = y$ has occurred when executing test case $test_x$. After 1,024 shots, the ensembles of the variable Y_R are given as $\{(0|0, 528), (1|0, 0), (2|0, 0), (3|0, 496)\}$. Then, its approximate probability distribution is $p_{Y_R|X}(y|0) : 0.5156, 0, 0, 0.4844$. Give the corresponding expected probability distribution $p_{Y_E|X}(y|0) : 0.5, 0, 0, 0.5$ and a threshold $v_T = 0.015$. The J-S divergence is calculated as $\text{JSD}(Y_E||Y_R|X) = 0.0110$, and the relation $\text{JSD}(Y_E||Y_R|X) < v_T$ holds. This implies that the above two probability distributions are approximately identical, so the patched version passes the test $test_0$.

4.3.2 Operator Estimation. In practice, the final state $|\psi_d^{(x)}\rangle$ of QP_D under the measurement-based testing process is scarcely acquired directly. On a real quantum computer, the final state can generally be inferred from the measurement results; in other words, a simple estimated state $|\hat{\psi}_d^{(x)}\rangle$ in place of the real state $|\psi_d^{(x)}\rangle$ participates in the subsequent analysis. Then, the estimated operator relating to $|\varphi^{(x)}\rangle$ and $|\hat{\psi}_d^{(x)}\rangle$ is abstracted as $\hat{U}_d \in \mathcal{L}(\mathcal{H}_i, \hat{\mathcal{H}}_d)$ that

$$|\hat{\psi}_d^{(x)}\rangle = \hat{U}_d |\varphi^{(x)}\rangle, \quad (9)$$

where $\hat{\mathcal{H}}_d$ is determined by the linear span of k_E estimated states that

$$\hat{\mathcal{H}}_d \equiv \text{span} \left(\left\{ |\hat{\psi}_d^{(x)}\rangle \right\}_{x=0}^{k_E-1} \right). \quad (10)$$

Actually, $\hat{U}_d$ does aid in giving a rational estimation of U_d during patch generation, which will be discussed in detail in Section 4.4.

The error operator Δ_d is introduced to quantify the relative error between $\hat{U}_d$ and U_d attributed to the estimated values not identical to the actual values. Δ_d is defined as

$$\Delta_d \equiv \left(\hat{U}_d - U_d \right) U_d^{-1} = \hat{U}_d U_d^{-1} - I. \quad (11)$$

Acceptable estimated operators do not necessarily follow a strict condition that $\Delta_d = O$, as long as the measurement results of QP_p meet program specifications. The linear map from the estimated operator to the real one can be modeled by

$$U_d = (\Delta_d + I)^{-1} \hat{U}_d. \quad (12)$$

If multiple estimated operators exist, then this form brings convenience to connecting them by the consistent operator U_d , which indicates the actual evolution phase of QP_D .

In general, the process of automatic quantum program repair is mainly considered as the transformation from U_d to U_e by a patch, so a unitary operator U_p can serve as the patch to perform the required linear mapping. Since it is possible to estimate and infer the state before and after the evolution part, U_p can be classified into two modes, Pre-repair patch U_{ep} and Post-repair patch U_{op} , which are separately located at above two positions.

Definition 1 (Unitary operator as the patch).

- *Pre-repair patch:* The unitary operator $U_{ep} \in \mathcal{L}(\mathcal{H}_i, \mathcal{H}')$ is introduced in the QP just before the qubit evolution, where $\mathcal{H}'$ is proposed as an intermediate space subjected to $U_d \in \mathcal{L}(\mathcal{H}', \mathcal{H}_e)$. It is expected that the patched version QP_p satisfies $U_r = U_d U_{ep}$.
- *Post-repair patch:* The unitary operator $U_{op} \in \mathcal{L}(\mathcal{H}_d, \mathcal{H}_e)$ is introduced in the QP just after the qubit evolution. It is hoped that the patched version QP_p satisfies $U_r = U_{op} U_d$.

Both operators aim to achieve the same measurement results for QP_p and QP_E . Nonetheless, the solution of U_p should depend on $\hat{U}_d$ and U_e rather than U_d . Then, two patches can be defined as

$$\begin{aligned} U_{ep} &\equiv \hat{U}_d^{-1} U_e, \\ U_{op} &\equiv U_e \hat{U}_d^{-1}. \end{aligned} \quad (13)$$

4.3.3 Patch Calculation. Supposed that the collection of orthonormal basis $\{|\varphi^{(x)}\rangle\}_{x=0}^{k_E-1}$ is arranged and k_E equations with x numbered from 0 to $k_E - 1$ are formulated based on Equation (4), the outer product operation can be implemented for each equation that

$$|\hat{\psi}_d^{(x)}\rangle\langle\varphi^{(x)}| = \hat{U}_d |\varphi^{(x)}\rangle\langle\varphi^{(x)}|. \quad (14)$$

The process of adding up k_E equations results in the following outcome:

$$\sum_{x=0}^{k_E-1} |\hat{\psi}_d^{(x)}\rangle\langle\varphi^{(x)}| = \hat{U}_d \sum_{x=0}^{k_E-1} |\varphi^{(x)}\rangle\langle\varphi^{(x)}|. \quad (15)$$

For convenience, the above operations can be abbreviated as

$$\hat{\Psi}_d \equiv \sum_{x=0}^{k_E-1} |\hat{\psi}_d^{(x)}\rangle\langle\varphi^{(x)}|, \quad \Psi_e \equiv \sum_{x=0}^{k_E-1} |\psi_e^{(x)}\rangle\langle\varphi^{(x)}|, \quad \Phi \equiv \sum_{x=0}^{k_E-1} |\varphi^{(x)}\rangle\langle\varphi^{(x)}|. \quad (16)$$

Supposed the Hermitian operator Φ is invertible, $\hat{U}_d$ can be expressed as

$$\hat{U}_d = \hat{\Psi}_d \Phi^{-1}. \quad (17)$$

In the same way, the evolution operator U_e is calculated as

$$U_e = \Psi_e \Phi^{-1}. \quad (18)$$

Then, referring to the definitions (13), two types of patches are written as follows:

– For *Pre-repair*,

$$U_{ep} = \Phi \hat{\Psi}_d^{-1} \Psi_e \Phi^{-1}. \quad (19)$$

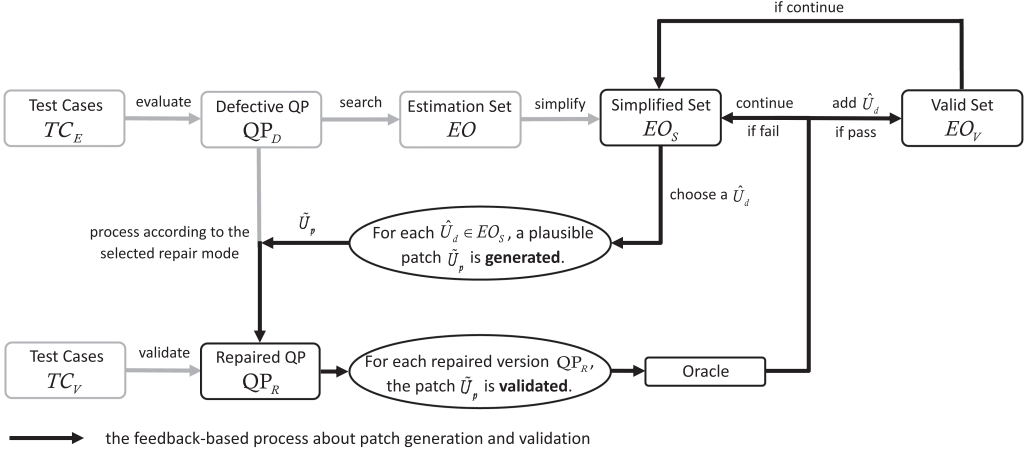


Fig. 4. The iterative generate-and-validate repair process with the patch $\tilde{U}_p$, which is generated through the corresponding estimated operator by searching and validated by testing the patched version.

— For *Post-repair*,

$$U_{op} = \Psi_e \hat{\Psi}_d^{-1}. \quad (20)$$

The assumption of $k_E = \dim(\mathcal{H}_i) = 2^n$ and orthogonality $\langle \varphi^{(x)} | \varphi^{(x')} \rangle = \delta_{x,x'}$ actually supports and simplifies the expression like Equation (19). Explicitly, the Hermitian operator Φ equals the unit operator I (more details are shown in Lemma A.1), where *the invertible operation is definitely guaranteed*. In the case of $\{|\varphi^{(x)}\rangle\}_{x=0}^{2^n-1}$ as the collection of orthogonal basis, Equation (19) can be simplified as

$$U_{ep} = \hat{\Psi}_d^{-1} \Psi_e. \quad (21)$$

The algebraic form like $\hat{\Psi}_d$ and Ψ_e naturally connects the inputs and outputs, consistent with the black-box model of the QP. Additionally, this form avoids inverting the operators so that evolution operators can be definitely calculated, regardless of initial states. Proposition 1 further discusses the properties of this form.

PROPOSITION 1. *For an n -qubit QP, the estimated operator is defined as $\hat{U} \equiv \Psi \equiv \sum_{x=0}^{k_E-1} |\psi^{(x)}\rangle \langle \varphi^{(x)}|$, where $\{|\varphi^{(x)}\rangle\}_{x=0}^{k_E-1}$ forms a collection of orthogonal bases.*

(I) $\hat{U}$ turns out to be a unitary operator only if $k_E = 2^n$.

(II) The rank of $\hat{U}$ follows $\text{rank}(\hat{U}) = k_E$.

PROOF. See Appendix B.4. □

To obtain a unitary operator $\hat{U}$, k_E is anticipated to be 2^n . The same can be illuminated for the second assumption in Section 4.2 that if there exist(s) m ($1 \leq m \leq n$) unmeasured qubit(s), 2^m equations like Equation (4) will be missed. Because of $\text{rank}(\hat{U}) = 2^n - 2^m$, $\hat{U}$ is implied to be irreversible.

Under certain circumstances, the calculated patch U_p cannot be considered a legal patch until it is corrected into a unitary one $\tilde{U}_p$ in higher precision. More details will be mentioned in Section 4.5.

4.4 Patch Generation and Validation

This section focuses on how to search appropriate patches according to the test results from QP_D . Figure 4 illustrates the repairing process in detail. Estimating evolution operators plays

an important role in calculating the alternative patches. Regardless of the selected repair mode, one estimation $\hat{U}_d$ uniquely reflects to a patch $\tilde{U}_p$, i.e., there exists a linear map $(U_e, \hat{U}_d) \mapsto \tilde{U}_p$. Considering multiple possibilities of estimated operators, a set named EO is proposed to contain all the possible $\hat{U}_d$. For the sake of the low cost of patch validation, it is necessary to reduce the size of EO into the simplified set EO_S i.e., $|EO| > |EO_S|$. Nonetheless, which one in EO has the same effect as U_d in terms of measurement still requires running and validating QP_p . The iterative process is proposed to generate and validate plausible patches, where the oracle judges whether the repair results are valid. With regard to the fixed version with $\hat{U}_d$, the estimated operators are added to the valid set EO_V ; otherwise, the process will repeat to choose another estimation.

4.4.1 Preliminary Estimation. Previous studies theoretically investigated specific methods about *quantum state reconstruction* (or *quantum tomography*) such as by maximum likelihood estimation [24], Bayesian [37], and machine learning [33]. However, due to the repair requirement, taking substantial resources to reconstruct quantum states precisely is unnecessary. Instead, if the repair procedure ensures the existence of plausible or correct patches in the search space, then a simple way to estimate quantum states with a focus on patches can be attempted.

In fact, it is possible to get a roughly estimated state $|\hat{\psi}_d\rangle$ via the probability distribution in the real vector space. With regard to Proposition A.1, when $D_d^{(x)}$ acting on $|\psi_d^{(x)}\rangle \in \mathcal{H}_d^{\mathbb{R}}$, $D_d^{(x)}$ can be regarded as the linear map in $\mathcal{L}(\mathcal{H}_d^{\mathbb{R}}, \mathcal{H}_d)$ and $(D_d^{(x)})^{-1}$ performs as the inverse one. Then, replacing $|\psi_d\rangle \in \mathcal{H}_d$ with an appropriate $|\hat{\psi}_d\rangle \in \hat{\mathcal{H}}_d^{\mathbb{R}} \subset \mathcal{H}_d$ may also guarantee the same measurement results. In this case, the estimated space is totally consistent with a real vector space that $\hat{\mathcal{H}}_d^{\mathbb{R}} = \hat{\mathcal{H}}_d$.

Now a preliminary estimation is given in $\hat{\mathcal{H}}_d^{\mathbb{R}}$, i.e., $\{|\hat{\psi}_{d0}^{(x)}\rangle\}_{x=0}^{k_E-1}$. Considering the unnormalized error that $\sum_y p_{Y_D|X}(y|x)$ may not strictly equal 1 caused by the floating-point error, the preliminary estimation can be defined as

$$|\hat{\psi}_{d0}^{(x)}\rangle \equiv \frac{[\sqrt{p_{Y_D|X}(0|x)}, \sqrt{p_{Y_D|X}(1|x)}, \dots, \sqrt{p_{Y_D|X}(2^n-1|x)}]^T}{\sqrt{\sum_{y=0}^{2^n-1} p_{Y_D|X}(y|x)}}. \quad (22)$$

It is verified that any preliminary estimation of final states satisfies $\langle \hat{\psi}_{d0}^{(x)} | \hat{\psi}_{d0}^{(x)} \rangle = 1$. Then, corresponding preliminary estimated operator $\hat{U}_{d0} \in \mathcal{L}(\mathcal{H}_i, \hat{\mathcal{H}}_d^{\mathbb{R}})$ can be calculated according to $\{|\varphi^{(x)}\rangle\}_{x=0}^{k_E-1}$ and $\{|\hat{\psi}_{d0}^{(x)}\rangle\}_{x=0}^{k_E-1}$.

4.4.2 Operator Searching. Apparently, the above process hardly ensures the orthogonality that $\langle \hat{\psi}_{d0}^{(x)} | \hat{\psi}_{d0}^{(x')} \rangle = \delta_{x,x'}$ especially for $x \neq x'$, which can be ascribed to the unconfirmed *phase* of each amplitude. In detail, the phase is denoted to the angle $\theta \in [0, 2\pi)$ of $e^{i\theta}$ in the complex plane, and any complex number $u \in \mathbb{C}$ can be written as $u = |u|e^{i\theta}$. For example, the preliminary estimated states for two states $|\psi_d^{(0)}\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, $|\psi_d^{(1)}\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ are identical that $|\hat{\psi}_{d0}^{(0)}\rangle = |\hat{\psi}_{d0}^{(1)}\rangle = |\hat{\psi}_d^{(0)}\rangle$ with all the phases being 0 ($e^{i\theta} = 1$), which violates the orthogonality that $\langle \hat{\psi}_{d0}^{(0)} | \hat{\psi}_{d0}^{(1)} \rangle = 1 \neq 0$.

From the perspective of qubit evolution, U_d is identified as a unitary operator without doubt, but $\hat{U}_{d0}$ may not be ensured. The phases are dictated to be logical so that $\hat{U}_{d0}$ can be modulated to the unitary operator $\hat{U}_d$. Therefore, the search space³ of estimated operators $\mathcal{S}_d$ can be determined

³The conception of search space is usually applied in computer science and software engineering, where the meaning of space is different from that in mathematics. Actually, the former indicates a denumerable and discrete set of certain states, while the latter stresses the property of continuity and possibly infinite dimensions. In this article, the search space merely corresponds to a set, the same as the previous studies about program repair.

as the identification of phases when the collection $\{|\hat{\psi}_0^{(x)}\rangle\}_{x=0}^{k_E-1}$ is given, such that

$$\mathcal{S}_d \equiv \left\{ \hat{U}_{d0} \circ \Theta \mid \Theta = \left(e^{i\theta_{m,m'}} \right)_{2^n \times 2^n}, \theta_{m,m'} \in [0, 2\pi) \right\}, \quad (23)$$

where Θ represents the phase matrix to modify the phase of each element in $\hat{U}_{d0}$ and the notation $\circ$ is *Hadamard product*, that, for instance, $[\sqrt{3}/2, 1/2]^\dagger \circ [e^{\pi i/3}, e^{2\pi i/3}]^\dagger = [\sqrt{3}e^{\pi i/3}/2, e^{2\pi i/3}/2]^\dagger$. As for a real search space $\mathcal{S}_d^{\mathbb{R}}$, any phase meets $\theta_{m,m'} \in \{0, \pi\}$ and the space is expressed as

$$\mathcal{S}_d^{\mathbb{R}} = \left\{ \hat{U}_{d0} \circ \Theta \mid \Theta \in \{1, -1\}^{2^n \times 2^n} \right\}. \quad (24)$$

Clearly, searching out a rational $\hat{U}_d$ means that corresponding patch U_p is generated referring to Equations (19) and (20). In other words, searching for U_p depends on whether at least one unitary operator $\hat{U}_d \in \mathcal{S}_d$ exists. For simplification, in this article, the precondition for the subsequent procedure of program repair is given that *the selected search space is the real one $\mathcal{S}_d^{\mathbb{R}}$* with finite alternative unitary operators $\hat{U}_d$, because it is unrealistic to validate tremendous patches. Generally, as $\mathcal{S}_d^{\mathbb{R}}$ is determined, states in $\mathcal{S}_d^{\mathbb{R}}$ can be visited and verified by order. Providing that the visited operator is identified as unitary, it can be stored in the estimated operator set EO . The whole procedure repeats until the searching steps reach the presupposed budget *budget* where $0 < b \leq |\mathcal{S}_d^{\mathbb{R}}|$. Ultimately, EO can be totally formed where $|EO| \leq b$.

However, traversing all the states in the search space intensively increases time costs and even seems unnecessary in most situations. In detail, there are $2^{2^n \cdot 2^n} = 2^{4^n}$ possible operators in $\mathcal{S}_d^{\mathbb{R}}$, so when n increases, restrictive conditions need to be proposed to avoid visiting trivial cases. To facilitate the analysis of searching, the sign operator S is introduced by Definition 2, and Example 6 explains how S works.

Definition 2 (Sign operator). The sign operator S acting on the state vector $|\psi\rangle = [\alpha_0, \alpha_1, \dots]^\dagger$ to conduct the mapping as

$$S(|\psi\rangle) \equiv [\text{sgn}(\text{Re}(\alpha_0)) + \text{sgn}(\text{Im}(\alpha_0))i, \text{sgn}(\text{Re}(\alpha_1)) + \text{sgn}(\text{Im}(\alpha_1))i, \dots]^\dagger, \quad (25)$$

where $\text{sgn}(x)$ is the well-known sign function, i.e.,

$$\text{sgn}(x) \equiv \begin{cases} -1, & x < 0 \\ 0, & x = 0 \\ 1, & x > 0 \end{cases}. \quad (26)$$

Example 6. Given the state vector as

$$|u\rangle = \frac{\sqrt{15}}{5}i|0\rangle + \frac{\sqrt{5} - \sqrt{5}i}{5}|1\rangle,$$

it is calculated that $S|u\rangle = [i, 1 - i]^\dagger$ by the definition.

Bringing in the sign operator S instead of the sign function $\text{sgn}(\cdot)$ is ascribed to its conformity to the operation of real matrices in programming languages, which retains the complete sign information. For better application of S , Proposition A.2 summarizes several of its properties. Proposition 2 is proposed to promote the simplification irrelevant to specific $\hat{U}_{d0}$ by eliminating the cases impossible to be unitary.

PROPOSITION 2. Given a collection of orthonormal bases $\{|u^{(m)}\rangle\}_{m=0}^{2^n-1}$, when S operates on that, for any $m \neq m'$, $S(|u^{(m)}\rangle)$ is linearly independent of $S(|u^{(m')}\rangle)$.

PROOF. See Appendix B.5. □

PROPOSITION 3. *Toward the case of n qubits, Proposition 2 can reduce the maximum searching possibilities $\max_{\hat{U}_{d0}} |\mathcal{S}_d^{\mathbb{R}}|$ from 2^{4^n} to $2^{2^n} \cdot (2^{2^n-1})! / (2^{2^n-1} - 2^n)!$.*

PROOF. See Appendix B.6. □

For each unitary operator $\hat{U}_d$ searched in $\mathcal{S}_d^{\mathbb{R}}$, it can be determined by a suitable real phase operator Θ in which there holds that $\hat{U}_d = \hat{U}_{d0} \circ \Theta$ and $\Theta = S\hat{U}_d$. Proposition 2 hints that if any two column vectors in Θ' are linear dependent, then it is impossible to generate a unitary operator when applying Θ' whatever $\hat{U}_{d0}$ is. Proposition 3 significantly demonstrates the effective simplification with the guideline Proposition 2. Supposed $n = 3$, for example, the relative simplification rate can reach

$$\left[\frac{1}{2^{4^n}} \cdot \left(2^{4^n} - \frac{2^{2^n} \cdot (2^{2^n-1})!}{(2^{2^n-1} - 2^n)!} \right) \right]_{n=3} = 1 - \frac{2^8 \times 2^7!}{2^{64} \times (2^7 - 2^3)!} = 0.2.$$

However, random search adopted as the first study on generating patches for QPs is inefficient because of the vast search space. The search algorithm is expected to include more effective heuristic information to instruct the process in future research.

4.4.3 Operator Equivalence. Let $\hat{U}_d^{(z)} \in EO$ be the z th estimated operator participating in the program repair. Furthermore, it should be noted that the selection of alternative operators in EO is subject to a specific probability distribution, where the random variable Z can be utilized to represent this process. Actually, two patches $\hat{U}_p^{(z)}$ and $\hat{U}_p^{(z')}$ ($z \neq z'$) generated by $\hat{U}_d^{(z)}$ and $\hat{U}_d^{(z')}$ respectively probably pose the *equivalent* effects when repairing QP_D , where Definition 3 clarifies how a couple of estimated operators are identified as equivalent. Then, before patch validation, such a couple of operators can be merged into any one of two, which contributes to the validation repeating fewer times.

Definition 3 (Operator equivalence). The estimated operator $\hat{U}_d^{(z)}$ is said to be *equivalent* to $\hat{U}_d^{(z')}$ if for $\hat{U}_d^{(z)}$ and $\hat{U}_d^{(z')}$ there holds

$$\forall x \in \{0, 1, \dots, k_V - 1\}, \forall y \in \{0, 1, \dots, 2^n - 1\} : p_{Y_R|X,Z}(y|x, z) = p_{Y_R|X,Z}(y|x, z'),$$

where $p_{Y_R|X,Z}(y|x, z)$ is the conditional probability of getting result $Y_R = y$ under the condition of the input $X = x$ or $|\varphi^{(x)}\rangle$ and the estimated operator $Z = z$ or $\hat{U}_d^{(z)}$.

For any $z \in \{0, 1, \dots, |EO| - 1\}$, $U_d = (\Delta_d^{(z)} + I)^{-1} \hat{U}_d^{(z)}$ holds, in which the superscript (z) of the error operator indicates its reliance on z . Then, for different values of Z that $Z = z$ and $Z = z' (z \neq z')$, they meet that

$$\left(\Delta_d^{(z)} + I \right)^{-1} \hat{U}_d^{(z)} = \left(\Delta_d^{(z')} + I \right)^{-1} \hat{U}_d^{(z')}.$$

Suppose

$$U_{\Delta}^{(z,z')} \equiv \hat{U}_d^{(z)} \left(\hat{U}_d^{(z')} \right)^{-1}, \quad (27)$$

where $U_{\Delta}^{(z,z')}$ is used to quantify the divergence between $\hat{U}_d^{(z)}$ and $\hat{U}_d^{(z')}$ by linear mapping. Then, it can also be expressed as

$$U_{\Delta}^{(z,z')} = \left(\Delta_d^{(z')} + I \right)^{-1} \left(\Delta_d^{(z)} + I \right). \quad (28)$$

In addition, it is evident that $U_{\Delta}^{(z,z')}$ is regarded a unitary operator that

$$\left(U_{\Delta}^{(z,z')} \right)^{-1} = U_{\Delta}^{(z',z)} = \left(U_{\Delta}^{(z,z')} \right)^{\dagger}.$$

ALGORITHM 1: Estimated Operator Calculation

Input: number of qubits n , preliminary estimated operator $\hat{U}_{d0}$, budget of searching steps $budget$
Output: the simplified set EO_S

```

1  $EO \leftarrow \emptyset$ ;
2  $PS \leftarrow \text{phase\_operators}(n)$ ;           // generate phase operators due to Proposition 2
3  $S_d^{\mathbb{R}} \leftarrow \text{search\_space}(\hat{U}_{d0}, PS)$ ;           // form the search space by (24)
4  $t \leftarrow 0$ ;                               // the search step
5 /* calculation of  $EO$  */
6 while  $t \leq budget$  do
7    $t \leftarrow t + 1$ ;
8    $\hat{U}_d \leftarrow \text{random\_selection}(S_d^{\mathbb{R}})$ ;           // randomly choose an operator from  $S_d^{\mathbb{R}}$ 
9    $S_d^{\mathbb{R}} \leftarrow S_d^{\mathbb{R}} - \{\hat{U}_d\}$ ;
10  if  $\hat{U}_d$  is unitary to certain precision then
11     $EO \leftarrow EO \cup \{\hat{U}_d\}$ ;           //  $\|\hat{U}_d^\dagger \hat{U}_d - I\| < \varepsilon$  upon the set precision  $\varepsilon$ 
12  end
13 end
14 /* simplifying  $EO$  into  $EO_S$  */
15 for  $\hat{U}_d^{(z)}, \hat{U}_d^{(z')} \in EO$  do
16   if  $\hat{U}_d^{(z)}, \hat{U}_d^{(z')}$  are equivalent due to proposition 4 then
17      $EO \leftarrow EO - \{\hat{U}_d^{(z')}\}$ ;           // merge the equivalent estimated operators
18   end
19 end
20  $EO_S \leftarrow EO$ ;
21 return  $EO_S$ ;

```

Using the operator $U_{\Delta}^{(z, z')}$, Proposition 4 proposes an algebraic view to determine a couple of equivalent operators in a direct way.

PROPOSITION 4. *The estimated operators $\hat{U}_d^{(z)}$ and $\hat{U}_d^{(z')}$ ($z \neq z'$) are equivalent if $U_{\Delta}^{(z, z')}$ is a scalar matrix, i.e., $U_{\Delta}^{(z, z')} = e^{\theta i} I$, where $\theta \in [0, 2\pi)$.*

PROOF. See Appendix B.7. □

The whole process of calculating the alternative estimated operators is briefly shown in Algorithm 1. The process outputs the simplified set EO_S with no equivalent operators deriving from EO .

4.4.4 Patch Validation. Generally, the repair process might produce two possible outcomes:

- *Fixed program:* the patched version QP_R obeys program specifications, thereby no failures produced by the version anymore.
- *Faulty program:* the patched version QP_R remains defective, since at least one test case does not pass during the test phase.

How to discriminate two outcomes requires concentration on patch validation that verifies the effectiveness of patches. Referring to Equation (7), the output distinction between QP_E and QP_D can be indicated by J-S divergence as $\text{JSD}(Y_E || Y_R | X, Z)$, where Y_E is independent of Z , so $p_{Y_E | X, Z}(y | x, z) = p_{Y_E | X}(y | x)$. For each patched version with $\hat{U}_d \in EO_S$, that all the tests pass means the validity of repair. Thus, to give an integrated view among all the possible values of x

and then eliminate the influence of variable X , $\text{JSD}_{\max}(Y_E||Y_R|Z)$ is defined based on the worst test result with the most significant divergence, such that

$$\text{JSD}_{\max}(Y_E||Y_R|Z) \equiv \max_x \text{JSD}(Y_E||Y_R|X, Z). \quad (29)$$

To be specific, given the threshold v_T , the version with $\hat{U}_d^{(z)}$ is identified as a fixed one if the relation $\text{JSD}_{\max}(Y_E||Y_R|Z) < v_T$ holds and vice versa. Intuitively, the best-patched version eventually accepted is the one with the test results closest to the expectation. In regards to validation, an assessment is made on which operator $\hat{U}_d^{(z)} \in EO_S$ contributes to producing a suitable patch that can achieve a fixed version. Such $\hat{U}_d^{(z)}$ can be included in the valid set EO_V that is defined as

$$EO_V \equiv \left\{ \hat{U}_d^{(z)} \mid \exists z : \text{JSD}_{\max}(Y_E||Y_R|Z = z) < v_T, \hat{U}_d^{(z)} \in EO_S \right\}, \quad (30)$$

where $EO_V \subseteq EO_S \subset EO$.

4.5 Unitary Correction

When running the program, some disturbances introduced accidentally may alter the theoretically unitary patch into a non-unitary one, so unitary correction is proposed to remove the negative impact via an algebraic procedure. The deviation to a theoretical patch can be ascribed to the randomness during measurement and the floating-point errors. First introduced by Luo et al. [38], any operator identified as a non-unitary one will result in the simulator error and then cause program crash.

To further analyze, let $\tilde{U}_p^{(z)}$ be a strict unitary operator that $U_p^{(z)}$ is expected to be, in which introducing the superscript (z) specifies the patch's reliance on the estimated operator $\hat{U}_d^{(z)}$, and $E_p = [|e_p^{(1)}\rangle, |e_p^{(2)}\rangle, \dots, |e_p^{(2^n)}\rangle]$ be a $2^n \times 2^n$ random complex matrix with small values denoted to the deviation, whose norm $\|E_p\|$ extremely closes to 0, where *the notion $\|\cdot\|$ in this article particularly refers to the 2-norm*. Then, $U_p^{(z)}$ is denoted as one unitary matrix added with a small deviation,

$$U_p^{(z)} = \tilde{U}_p^{(z)} + E_p. \quad (31)$$

Given a machine precision of ε_m insides APIs, the objective is to transform $U_p^{(z)}$ into a valid patch $\tilde{U}_p^{(z)}$ by reducing *the error of unitarity* $\varepsilon_u^{(z)}$. This error, defined as follows, represents the deviation from $\tilde{U}_p^{(z)}$ to the unitary operator in theory,

$$\varepsilon_u^{(z)} \equiv \left\| \left(\tilde{U}_p^{(z)} \right)^\dagger \tilde{U}_p^{(z)} - I \right\|. \quad (32)$$

Besides, *the error of similarity* $\varepsilon_s^{(z)}$, defined as the distance between $\tilde{U}_p^{(z)}$ and $U_p^{(z)}$, is proposed to quantify their similarity before and after correction, whereby

$$\varepsilon_s^{(z)} \equiv \left\| \tilde{U}_p^{(z)} - U_p^{(z)} \right\|. \quad (33)$$

Intuitively, two directions of unitary correction can be summarized as follows:

- *The error of unitarity* $\varepsilon_u^{(z)}$: It is expected to control $\varepsilon_u^{(z)}$ within the specified error range of APIs, such that the program can be prevented from the crash triggered by the non-unitary errors.
- *The error of similarity* $\varepsilon_s^{(z)}$: It is also essential to make sure $\varepsilon_s^{(z)}$ as small as possible; otherwise, the patches are modified to the ones not comforting to the requirement of program repair.

In linear algebra, QR decomposition disparts a complex non-singular matrix A into a unitary matrix Q and an upper triangular matrix R through the product $A = QR$. According to Proposition A.3, it is dictated that U_p is not necessarily unitary but definitely invertible, which infers that it is a feasible and unique procedure through QR decomposition to transform $U_p^{(z)}$ into the unitary $\tilde{U}_p^{(z)}$, referred to Lemma A.3. Then, the decomposition of $U_p^{(z)}$ is taken into account,

$$U_p^{(z)} = Q_p^{(z)} R_p^{(z)}. \quad (34)$$

There are several typical methods for how to implement QR decomposition, such as the *Gram-Schmidt process*, *Householder transformations*, and *Givens rotations*, where compared to the others, each method stands out in one aspect like low computing complexity, high numerical stability⁴ and so on. The theoretical outcomes of decomposition may vary in different methods, especially for signs of each element in the unitary matrix $Q_p^{(z)}$. In other words, simply applying QR decomposition can merely decrease the error of unitarity $\varepsilon_u^{(z)}$ close to the machine precision, which hardly guarantees the low error of similarity $\varepsilon_s^{(z)}$ instead, where Example 7 interprets the phenomena in detail.

Example 7. Supposed that the estimated operator $\hat{U}_d^{(0)}$ is chosen to generate the patch, and $U_p^{(0)}$ is given in a low precision as follows, $U_p^{(0)}$ will be rectified into one of the unitary operators *in theory* (it is unrealistic to obtain and even display the absolutely accurate numerical value of $\sqrt{2}/2$), such as $\tilde{U}_p^{(0)}$ and $\tilde{U}_{p'}^{(0)}$ referred to two different methods of QR decomposition. However, the error of similarity for $\tilde{U}_{p'}^{(0)}$ that $\varepsilon_{s'}^{(0)} = 1.998$ is prominently greater than the one for $\tilde{U}_p^{(0)}$ that $\varepsilon_s^{(0)} = 1.5101 \times 10^{-4}$, which implies that the so-called correction into $\tilde{U}_{p'}^{(0)}$ totally spoils the process of generating the expected patch. Hence, the phase (or the sign for the real operator) of each element in the patch should be manipulated to match the acceptable one $\tilde{U}_p^{(0)}$,

$$U_p^{(0)} = 0.707 \begin{bmatrix} 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \\ 1 & 1 & 0 & 0 \end{bmatrix}, \tilde{U}_p^{(0)} = \frac{\sqrt{2}}{2} \begin{bmatrix} 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \\ 1 & 1 & 0 & 0 \end{bmatrix}, \tilde{U}_{p'}^{(0)} = \frac{\sqrt{2}}{2} \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 1 & 1 \\ 1 & -1 & 0 & 0 \end{bmatrix}.$$

Therefore, to regulate the phases, the corrected patch $\tilde{U}_p^{(z)}$ can be expressed as follows with the help of the sign operator S mentioned in Definition 2,

$$\begin{aligned} \text{Re} \left(S Q_p^{(z)} \right) &= \text{Re} \left(S U_p^{(z)} \right) D_p^{(z)}, \\ \tilde{U}_p^{(z)} &= Q_p^{(z)} D_p^{(z)}. \end{aligned} \quad (35)$$

It seems uncommon to be barely concerned about the real parts of the operator. Virtually, when calculating with complex values, a possible case may be confronted with, for instance, that a real number $1.0000 + 0.0000i$ is stored as a complex number $1.0000 + 0.0001i$ due to the floating-point errors. However, as S acts on them, the results become utterly different due to S devoid of continuity, whereby $S(1.0000 + 0.0000i) = 1$, $S(1.0000 + 0.0001i) = 1 + 1i$. In intuition, the imaginary part should be too tiny to bring conspicuous influence, and that is why this part is ignored at last. In the above equation, the diagonal unitary operator $D_p^{(z)}$ is proposed to modulate the signs of $Q_p^{(z)}$, but may not be of full rank, such that it cannot be calculated directly through the invertible linear

⁴A method is called numerically stable only if the errors brought by the process of numerical operation tend to converge after executing certain steps.

transformation. Therefore, the following equation offers a convenient approach to solve $D_p^{(z)}$ by using $R_p^{(z)}$,

$$D_p^{(z)} = \text{Re} \left(S \left(R_p^{(z)} \circ I \right) \right). \quad (36)$$

This equation can be explained that $D_p^{(z)}$ is related to the real part of diagonal elements of $R_p^{(z)}$. More details about the mechanism of Equations (35) and (36) are demonstrated as proposition 5. It is promised that $\lim_{E_p \rightarrow 0} \epsilon_s^{(z)} = \lim_{E_p \rightarrow 0} \|\tilde{U}_p^{(z)} - U_p^{(z)}\| = 0$ and $\text{Re}(S(\tilde{U}_p^{(z)})) = \text{Re}(S(U_p^{(z)}))$, demonstrating that unitary correction will not transform patches into other unanticipated unitary operators.

PROPOSITION 5. *Supposed $U_p^{(z)}$ defined by Equation (31) as $U_p^{(z)}$ is corrected $U_p^{(z)}$ to $\tilde{U}_p^{(z)}$, the following propositions hold:*

- (I) $\lim_{E_p \rightarrow 0} (\tilde{U}_p^{(z)} - U_p^{(z)}) = 0$.
- (II) $\text{Re}(S(\tilde{U}_p^{(z)})) = \text{Re}(S(U_p^{(z)}))$.

PROOF. See Appendix B.8. □

4.6 Algorithm

The entire procedure of UnitAR is shown in Algorithm 2. To be brief, the defective program QP_D is tested to evaluate its divergence to the expected version QP_E . It is possible to determine the set of alternative estimated operators, known as EO_S , by analyzing the probability distribution from testing results. Then, each patch can be randomly generated by the selected operator in EO_S and validated by the test suite TS_V .

5 EMPIRICAL STUDIES

5.1 Research Questions

The empirical studies have centered on addressing the following three research questions:

- *RQ1: What are the repair results of quantum programs?*
This RQ compares the results of repairing QPs when respectively adopting the proposed APR approach (UnitAR) and two traditional APR approaches (GenProg and TBar), which have been widely applied in repairing CPs. More specifically, the comparison is based on the effectiveness and efficiency of program repair.
- *RQ2: Do the proposed propositions contribute to the efficiency of patch generation and validation?*

In Section 4.4, the article proposes Proposition 2 to reduce the search space regardless of the estimated operators and Proposition 4 to merge the equivalent operators that induce the same measurement results. The RQ aims to determine whether and how the application of these propositions can reduce the number of patches and shorten the execution time of the iterative procedure.

- *RQ3: How effective is the procedure of unitary correction?*
This RQ verifies the effect of the proposed unitary correction that transfers the non-unitary candidate patches into the desired unitary ones. In the experiment, the numerically stable method, Householder reflections, is applied to implement QR decomposition, such that the procedure of numerical computing scarcely introduces additional errors.

5.2 Object Programs

This article chooses 34 defective QPs as the objects to be repaired, and all of them are written via Qiskit and based on Python. Among these objects, 29 programs are generated by the mutation

ALGORITHM 2: The Entire Procedure of UnitAR

Input: number of qubits n , budget of searching steps $budget$, defective quantum program QP_D , test suites TS_E and TS_V

Output: set of fixed quantum programs VR , set of valid estimated operators EO_V

```

1   $VR \leftarrow \emptyset$ ;
2   $EO_V \leftarrow \emptyset$ ;
3   $U_e \leftarrow \text{evolution\_operator}(TS_E.\text{initial\_states}, TS_E.\text{final\_states})$ ;           // refer to (17) or (18)
4  /* test the defective quantum program */
5  while  $TS_E \neq \emptyset$  do
6       $|\varphi^{(x)}\rangle, x \leftarrow \text{random\_selection}(TS_E.\text{initial\_states})$ ;
7       $TS_E \leftarrow TS_E - \{TS_E[x]\}$ ;
8       $\{p_{Y_D|X}(y|x)\}_{y=0}^{2^n-1} \leftarrow \text{run}(QP_D, |\varphi^{(x)}\rangle)$ ;           // run the quantum program
9       $|\hat{\psi}_{d0}^{(x)}\rangle \leftarrow \text{preliminary\_estimation}(\{p_{Y_D|X}(y|x)\}_{y=0}^{2^n-1})$ ;           // refer to (22)
10 end
11  $\hat{U}_{d0} \leftarrow \text{evolution\_operator}(\{|\varphi^{(x)}\rangle\}_{x=0}^{k_E-1}, \{|\hat{\psi}_{d0}^{(x)}\rangle\}_{x=0}^{k_E-1})$ ;
12  $EO_S \leftarrow \text{estimated\_operator}(\hat{U}_{d0}, budget)$ ;           // refer to Algorithm 1
13 /* generate and validate the patch */
14 while  $EO_S \neq \emptyset$  do
15      $\hat{U}_d^{(z)} \leftarrow \text{random\_selection}(EO_S)$ ;
16      $EO_S \leftarrow EO_S - \{\hat{U}_d^{(z)}\}$ ;
17      $M \leftarrow \text{random\_selection}(\{\text{Pre-repair}, \text{Post-repair}\})$ ;
18      $\tilde{U}_p^{(z)} \leftarrow \text{patch\_calculation}(U_e, \hat{U}_d^{(z)}, M)$ ;           // refer to (13)
19      $\tilde{U}_p^{(z)} \leftarrow \text{unitary\_correction}(\tilde{U}_p^{(z)})$ ;           // refer to (34) ~ (36)
20      $QP_R \leftarrow \text{program\_repair}(\tilde{U}_p^{(z)}, QP_D, M)$ ;           // introduce the patch
21     /* test the patched version */
22     while  $TS_V \neq \emptyset$  do
23          $|\varphi^{(x)}\rangle, x \leftarrow \text{random\_selection}(TS_V.\text{initial\_states})$ ;
24          $TS_V \leftarrow TS_V - \{TS_V[x]\}$ ;
25          $\{p_{Y_R|X}(y|x)\}_{y=0}^{2^n-1} \leftarrow \text{run}(QP_R, |\varphi^{(x)}\rangle)$ ;
26     end
27      $P_{Y_R} \leftarrow (p_{Y_R|X}(y|x))_{2^n \times k_V}$ ;           // store the probabilities
28     if  $\text{oracle}(P_{Y_R}, TS_V) == \text{TRUE}$  then
29          $VR \leftarrow VR \cup \{QP_R\}$ ;           // save if it is validated by the oracle
30          $EO_V \leftarrow EO_V \cup \{\hat{U}_d^{(z)}\}$ ;           // save the estimated operator
31     end
32 end
33 return  $VR, EO_V$ ;

```

operators, and the other 5 are the real-world buggy programs. For the former type of objects, the article adopts 5 original QPs from the primary optional sources mentioned in the existing study by Fortunato et al. [16]. By using the mutation operations of different kinds and diverse numbers, corresponding defective versions can be mutated from each of the bug-free QPs. For the latter type of objects, the article employs 5 available buggy QPs provided by the benchmark Bugs4Q [62], where the bug type of all the above 5 programs is marked with Output Wrong. There are brief introductions of the selected object programs.

- **Root of NOT (RON)**: This program demonstrates a construction of the operation ROOTNOT using h and p gates. The corresponding quantum circuit includes two ROOTNOT operations to constitute one NOT (x) operation.
- **Custom Conditional-phase (CCP)**: The program aims to establish a two-qubit controlled-phase gate that can rotate the phase to any value in the register.
- **Remote-controlled Randomness (RCR)**: The program includes two qubits, where measuring out one qubit will instantly affect the probability of the bit value derived from the other qubit. Moreover, the interaction between two qubits exists over any distance of space or time.
- **QFT**: The algorithm executed by this program is the quantum implementation of the discrete Fourier transform. Usually, this algorithm works as a crucial part of numerous famous quantum algorithms.
- **Entangled Qubits (EQ)**: The program yields a state in entanglement, and this kind of state is conventionally called a Bell pair. The specific quantum circuit of two-qubit version is shown in Figure 1.
- **Bugs4Q Database (Bugs4Q)**: The available database includes buggy programs along with the corresponding fixed versions. All the QPs are collected from the related issues or problems on the software development platforms, such as GitHub, StackOverflow, and Stack Exchange.

Virtually, the latent defects that lead to wrong outputs can be attributed to undesired gate applications. From the perspective of quantum gates, this article considers the following four kinds of concrete wrong operations, where they can be utilized as mutation operators or used to explain the real-world bugs.

- **Add gate (AG())**: Add quantum gates to the quantum circuit. If the gate h is seeded, then this operation can be denoted as $AG(h)$.
- **Delete gate (DG())**: Delete quantum gates from the quantum circuit. If both h and x are removed, then this operation can be denoted as $DG(h,x)$.
- **Replace gate (RG())**: Replace a quantum gate with another one. If the gate h is substituted by gate x , then this transformation can be indicated as $RG(h \rightarrow x)$.
- **Switch qubits (SQ())**: Switch qubits in a multiple-qubit gate. In the event of exchanging the control and target qubits in the controlled gate cx , this can be expressed as $SQ(cx)$.

Table 1 summarizes the basic information of the object programs, including the program names (Quantum programs), the number of qubits (# Qubits), the depth of quantum circuit (Depth), the number of defects (# Defects), and the specific unexpected gate operations (Undesired gate applications).

Referred to the Bugs4Q database, developers are inclined to try simple QPs for their preliminary quantum experience. However, human intuition is better fitted for the classical world than for quantum mechanics [19]. Hence, undesired bugs are still inevitable even in these simple programs. As a first attempt to automatically repair QPs, the article just employs simple programs with only a small number of qubits in the experiments. With increasing requirements of quantum programs, more complicated QPs should be considered and analyzed in future work.

5.3 Variables

5.3.1 Independent Variables. One of the independent variables is the baseline APR methods. The genetic algorithm GenProg and the template-based approach TBar are adopted as the baselines for RQ1, where more details will be discussed in Section 5.4. The other independent variable refers to two distinct states: before and after adopting the proposed schemes, such as the propositions for the generate-and-validate procedure (for RQ2) and the unitary correction for candidate patches (for RQ3).

Table 1. Basic Information on the Object Programs

QPs	# Qubits	Depth	# Defects	Undesired gate applications
RON-v1	1	9	1	AG(x)
RON-v2	1	7	1	DG(h)
RON-v3	1	10	2	AG(h, h)
RON-v4	1	6	2	DG(rz, rz)
RON-v5	1	6	3	RG(h→x), DG(rz, rz)
CCP-v1	2	9	1	RG(cx→ch)
CCP-v2	2	8	2	DG(h, h)
CCP-v3	2	9	2	DG(h), SQ(cx)
CCP-v4	2	9	3	RG(h→x, h→x, cx→ch)
CCP-v5	2	9	3	DG(h, h), AG(x)
RCR-v1	2	5	1	DG(h)
RCR-v2	2	5	2	DG(h), SQ(cx)
RCR-v3	2	6	2	SQ(cx), RG(t→x)
RCR-v4	2	6	3	RG(h→t, h→t, h→t)
RCR-v5	2	8	3	DG(h), AG(h, ch)
QFT-v1	3	8	1	RG(h→x)
QFT-v2	3	9	2	RG(h→x, h→x)
QFT-v3	3	7	2	DG(h, h)
QFT-v4	3	8	3	AG(h, h), DG(swap)
QFT-v5	3	8	4	DG(cp, cp, cp, h)
EQ-v1	2	4	1	AG(x)
EQ-v2	2	3	2	DG(h), SQ(cx)
EQ-v3	2	3	2	DG(cx), RG(h→x)
EQ-v4	2	5	3	RG(h→x), AG(h, z)
EQ-v5	2	4	3	AG(h), DG(h), SQ(cx)
EQ-v6	4	4	1	DG(cx)
EQ-v7	6	7	1	RG(h→x)
EQ-v8	8	8	1	SQ(cx)
EQ-v9	10	11	1	AG(h)
Bugs4Q-id8	3	4	2	SQ(crz, crz)
Bugs4Q-id25	3	5	1	SQ(ccx)
Bugs4Q-id26	2	4	1	RG(x→h)
Bugs4Q-id27	3	11	6	Reverse a sequence of quantum gates
Bugs4Q-id39	4	8	4	DG(h, h, h, h)

“-v” (e.g., “-v1”) indicates the mutation version of the original program. “-id” (e.g. “-id8”) corresponds to the bug ID in the benchmark Bugs4Q.

5.3.2 Dependent Variables. RQ1 considers the success rate of candidate patches and execution time of program repair for each APR method to evaluate the effectiveness and efficiency of corresponding methods. Apart from the execution time, RQ2 further investigates the sizes of sets ($|S_d^R|$, $|EO|$, $|EO_S|$, and $|EO_V|$) relevant to patches when adopting the propositions for optimization. This helps to determine whether the above step promotes the efficiency of program repair through

merging redundant equivalent patches. RQ3 focuses on the error of unitarity ε_u and the error of similarity ε_s to validate the procedure of unitary correction, where both kinds of errors should be controlled within a reasonable range.

5.4 Experimental Settings

5.4.1 Test Suites. The article adopts 2^n computational basis states $\{|x\rangle\}_{x=0}^{2^n-1}$ to consist of the common test set to sufficiently evaluate the defective program QP_D and validate the patched program QP_R , i.e., $TC_E = TC_V$ and $k_E = k_V = 2^n$. The program specification $|\psi_e^{(x)}\rangle$ corresponding to each test case $|\varphi^{(x)}\rangle$ can be obtained according to the functionalities of bug-free program versions before the mutation or the fixed version available in the database.

5.4.2 Baselines. The article selects two generate-and-validate approaches (i.e., GenProg and TBar) as the baselines. Consider that the two approaches are proposed for repairing CPs. Thus, it is necessary to make appropriate modifications for both approaches, such that they can probably repair QPs.

- GenProg: This approach incorporates mutation and crossover operators to generate candidate patches.
 - *Mutation*: In the following experiments, the mentioned four mutation operators (AG, DG, RG, and SQ) are used in the mutation part. According to the broadly acknowledged assumption that most defects can be repaired by applying existing code from another location in the program [14], candidate quantum gates corresponding to the above mutation operators can be chosen from those already employed in the buggy program. For example, a three-qubit buggy program includes a statement `qc.h(0)`, so the statements `qc.h(0)`, `qc.h(1)`, and `qc.h(2)` can be added to a certain location (AG operation) or used to replace an existing code snippet (RG operation).
 - *Crossover*: The crossover part refers to generating offspring (i.e., the new patched programs) by switching and combining the code snippets of parents (i.e., previous patched programs). The location where the crossover occurs is called a cutoff point, and this point can be randomly settled at a reasonable location of the parent program.
 - TBar: TBar generates a new patch by selecting a fix pattern and then mutating the suspicious code. Among the fix patterns introduced in Reference [32], the article opts for four kinds suitable to conduct mutation based on quantum gates. More specifically,
 - *Mutate literal expression*: This fix pattern can be used to mutate the parameter of a rotation gate, such as `qc.cx(math.pi, 0) → qc.cx(-math.pi, 0)`.
 - *Mutate method invocation expression*: It consists of four sub-fix patterns that are shown as follows.
 - *Replace name*: Replace the quantum gate in case of the same qubit (e.g., `qc.x(1) → qc.h(1)`).
 - *Replace argument*: Replace the qubit in case of the same quantum gate (e.g., `qc.x(1) → qc.x(2)`).
 - *Remove argument*: Delete one of the quantum gates if this kind of gate respectively acts on multiple qubits (e.g., `qc.x([0, 1]) → qc.x(0)`).
 - *Insert argument*: Add a quantum gate if this kind of gate has been employed (e.g., `qc.x(0) → qc.x([0, 1])`).
 - *Move statement*: Move the quantum gate to a random location in the event of acting on the same qubit.
 - *Remove statement*: Delete the quantum gate, which resembles the mutation operator DG.
- Each fix pattern has an equal probability of being chosen to mutate the buggy program.

5.4.3 Detailed Parameters. For the measurement-based procedure, the article simply adopts the default argument 1,024 as the number of shots. Even so, the statistical randomness poses an impact on identifying potential unitary operators during patch generation. To handle that, the article roughly treats $\hat{U}_d \in \mathcal{S}_d^{\mathbb{R}}$ as an approximately unitary one if $\|\hat{U}_d^\dagger \hat{U}_d - I\| < 0.1$ during the searching phase. Similarly, the identification of a scalar matrix is set in the relatively coarse criteria that $\|U_\Delta^{(z,z')} - e^{i\theta} I\| < 0.1$, when simplifying EO into EO_S . Actually, the procedure of unitary correction can guarantee the validity of generated patches, so the above low-precision settings do no harm to the repair results. Consider that the time consumption should be sufficient but affordable. For each independent repair procedure, the article sets the time limit as 2 hours and configures the budget as 100,000. That is, even if the search space is exceedingly large, only part of the operators in the search space is needed for the generate-and-validate procedure.

5.4.4 Execution Environment. The experiments are conducted through the classical simulator via Qiskit, where the version is installed with `qiskit-terra=0.25.2.1`. For the purpose of simulating a quantum system, the article chooses "qasm_simulator" as the backend. Furthermore, the Ubuntu 22.04.3 LTS x86 64 operating system along with a kernel of 6.2.0-36-generic and a CPU of Intel i9-10900K (20) @ 5.300 GHz is employed to execute the codes involved in the experiments.

5.5 Experimental Results

5.5.1 RQ1: Repair Results of APR Methods. This experiment compares the repair results when respectively adopting GenProg, TBar, and UnitAR. In this experiment, patches can be randomly generated from the search space and validated by running the patched version until the execution time reaches the limit.

As shown in Table 2, each of the APR methods can merely repair part of the buggy programs. Compared to the two baseline methods GenProg and TBar, the proposed method UnitAR manifests the highest fixed rate (69.70%). For most of the buggy programs that three APR methods can fix, UnitAR can reach the highest success rates while TBar has the lowest ones. The results imply that UnitAR can utilize the divergence between expected and tested outputs to construct a refined search space from the perspective of linear operation. Since only the unitary operators in the search space can be chosen as candidate patches, the cost of validating patches by running QPs can be reduced to some extent. By contrast, GenProg and TBar do not apply the outputs produced from a QP to build the search space, such that the size of the space may be extremely large. Compared to repairing a CP, whatever positive or negative tests almost cover all the wires of the quantum circuit corresponding to the buggy QP. Hence, it is difficult for the two traditional APR methods GenProg and TBar to determine possible buggy statements in the same way as the fault localization method in CPs. Additionally, TBar seems to have worse performance than GenProg, which contradicts the conclusions in the existing studies related to repairing CPs. In fact, the modified version of TBar employed in the article only incorporates a limited number of appropriate fix patterns that have been summarised in Reference [32], thereby confining the search space and excluding possible patches. Currently, it is challenging to employ traditional APR methods to repair QPs.

Apart from the effectiveness of program repair, UnitAR outperforms GenProg and TBar owing to less time consumption. Excluding QFT-v1, if UnitAR can repair a certain QP, then it will cost the least time to find a reasonable patch on average. In fact, the time consumption when applying an APR method may related to the success rates. Under the circumstance that plenty of desired patches cluster in a limited search space, it is trivial for an APR method to find at least one solution.

In terms of the programs to be patched, Bugs4Q-id8 outputs identical probability distributions to its bug-free version, so it cannot be distinguished as a buggy version based on the existing

Table 2. The Repair Results of 34 Buggy Versions in the Case of the "qasm_simulator" Backend

QPs	GenProg			TBar			UnitAR		
	Success	Time	Results	Success	Time	Results	Success	Time	Results
RON-v1	16.13%	0.06	✓	9.54%	0.05	✓	50.06%	0.01	✓
RON-v2	62.23%	0.02	✓	6.79%	0.06	✓	100.00%	0.01	✓
RON-v3	12.78%	0.08	✓	5.83%	0.08	✓	100.00%	0.01	✓
RON-v4	0.00%	N/A	✗	0.00%	N/A	✗	100.00%	0.01	✓
RON-v5	61.07%	0.02	✓	25.01%	0.02	✓	49.42%	0.01	✓
RCR-v1	0.88%	3.04	✓	0.00%	N/A	✗	3.98%	0.12	✓
RCR-v2	0.12%	22.22	✓	0.00%	N/A	✗	4.08%	0.11	✓
RCR-v3	0.00%	N/A	✗	0.00%	N/A	✗	52.80%	0.02	✓
RCR-v4	0.00%	N/A	✗	0.00%	N/A	✗	0.00%	N/A	✗
RCR-v5	0.19%	17.43	✓	0.00%	N/A	✗	0.50%	1.12	✓
CCP-v1	13.45%	0.26	✓	5.81%	1.58	✓	6.12%	0.13	✓
CCP-v2	0.00%	N/A	✗	0.00%	N/A	✗	0.00%	N/A	✗
CCP-v3	34.09%	0.11	✓	1.75%	3.56	✓	93.40%	0.02	✓
CCP-v4	2.11%	1.77	✓	0.00%	N/A	✗	1.36%	0.90	✓
CCP-v5	0.00%	N/A	✗	0.00%	N/A	✗	0.00%	N/A	✗
QFT-v1	43.59%	0.32	✓	0.21%	2.70	✓	0.29%	5.59	✓
QFT-v2	0.00%	N/A	✗	0.00%	N/A	✗	99.89%	0.12	✓
QFT-v3	0.00%	N/A	✗	0.00%	N/A	✗	1.54%	1.87	✓
QFT-v4	11.48%	1.33	✓	3.10%	0.24	✓	N/A	N/A	✗
QFT-v5	0.00%	N/A	✗	0.00%	N/A	✗	0.00%	N/A	✗
EQ-v1	6.87%	0.37	✓	13.76%	0.04	✓	90.79%	0.03	✓
EQ-v2	0.00%	N/A	✗	0.00%	N/A	✗	1.67%	0.45	✓
EQ-v3	0.00%	N/A	✗	0.00%	N/A	✗	1.66%	0.45	✓
EQ-v4	0.89%	3.34	✓	4.78%	0.08	✓	24.52%	0.05	✓
EQ-v5	15.97%	0.18	✓	0.00%	N/A	✗	25.28%	0.05	✓
EQ-v6	0.00%	N/A	✗	0.00%	N/A	✗	39.94%	0.47	✓
EQ-v7	0.00%	N/A	✗	0.00%	N/A	✗	N/A	N/A	✗
EQ-v8	0.00%	N/A	✗	0.00%	N/A	✗	N/A	N/A	✗
EQ-v9	0.00%	N/A	✗	0.00%	N/A	✗	N/A	N/A	✗
Bugs4Q-id8	—	—	—	—	—	—	—	—	—
Bugs4Q-id25	0.34%	39.34	✓	0.00%	N/A	✗	100.00%	0.12	✓
Bugs4Q-id26	0.00%	1200.00	✓	0.00%	N/A	✗	8.26%	0.09	✓
Bugs4Q-id27	0.00%	N/A	✗	0.00%	N/A	✗	0.00%	N/A	✗
Bugs4Q-id39	0.00%	N/A	✗	0.00%	N/A	✗	0.00%	N/A	✗
Average fixed	17 / 33 = 51.25%			10 / 33 = 30.30%			23 / 33 = 69.70%		

The header "Success" indicates the proportion of the generated candidate patches that enable the corresponding patched version to be validated as a fixed one. "N/A" in the "Success" column of "UnitAR" implies that no unitary operator is found in the subset with the maximum size of $budget = 100k$. The header "Time" means how many seconds are consumed on average to find a correct patch, where "N/A" implies that no such patch is found within the time limit (2h). The header "Results" refers to whether the buggy version can be fixed through the corresponding APR method. The last row of the table, "Average fixed", stands for the proportion of the fixed versions among all the buggy ones. Other than those, the rows with "—" indicate the corresponding programs are identified as bug-free ones when adopting the measurement-based test oracle.

measurement-based test oracle. Because the quantum measurement may overlook the phases of a quantum state, it raises a potential oracle problem for examining the behavior of QPs. For some of the other programs, like RCR-v4, CCP-v2, v5, QFT-v5, EQ-v7, v8, v9 and Bugs4Q-id27, id39, none of the three APR methods can repair them. This indicates that the difficulty of repairing QPs

automatically involves the specific quantum gates and the number of qubits. For example, for the other three versions of CCP, at least one APR method can repair them, while each APR method has trouble in fixing the mutation versions v2, v5. Moreover, the increase in qubits makes it harder to repair QPs automatically. When the qubits increase to no less than 6 (such as EQ-v7, v8, v9), no APR method can realize the repair of QPs, although the versions with fewer qubits still can be fixed by at least one APR method.

Answer to RQ1: For the 34 buggy programs (only 33 to be fixed), the proposed method UniTAR has the highest average fixed rate compared to the two baseline methods (51.25% for GenProg and 30.30% for TBar). Especially for most buggy QPs that the above three methods can fix, UniTAR outperforms GenProg and TBar in terms of the highest success rates and least execution time.

5.5.2 RQ2: Optimization for Search Phase. In the generate-and-validate procedure, Proposition 2 and Proposition 4 in Section 4.4 are proposed to decrease redundant patches and facilitate repair efficiency. To be specific, they aim to optimize the procedure respectively in the verification of unitarity and the consolidation of equivalent operators. This experiment only chooses the QPs whose sizes of search space (i.e., $|\mathcal{S}_d^{\mathbb{R}}|$) are no more than the budget. Each element of the search space is traversed to investigate how the step of optimization works in generating and validating candidate patches. For further research on two repair modes, this experiment also incorporates the results when only applying one repair mode to UniTAR. Besides, the repair procedure of each buggy program is repeated 50 times, such that the corresponding results should be statistically significant.

Table 3 shows the details about the sizes of sets and the time costs. For the size of search space $|\mathcal{S}_d^{\mathbb{R}}|$, the value is capped at 2^{4n} , like the maximum $|\mathcal{S}_d^{\mathbb{R}}|$ of 2-qubit RCR as $2^{16} = 65,536$. Nevertheless, the size of a set does not always reach such a tremendous number, owing to the value 0 of operators that reduce the possibilities of sign combinations. With regard to the simplification from EO to EO_S , the relation that $|EO| > |EO_S|$ holds for all the QPs. In detail, $|EO|$ approximates the twice of $|EO_S|$, which hints that for an estimated operator, there lies only one other operator equivalent to that. For the validation results, note that $|EO_V|$ of some QPs manifest a non-integer form. This implies that the test oracle may not be as deterministic as that in CPs. For example, $|EO_V|$ of RON-v1 with post-repair mode is 1.96 after optimization. This may indicate that in most of the 50 repeats, 2 appropriate patches can be verified, while in a few cases, there is only 1 patch that can fix the program. The fluctuation may relate to shots for measuring the final states and the configured threshold v_T involved in the test oracle.

For the execution time, in 19 of 25 cases (marked in bold), applying the propositions to both repair modes (i.e., pre-repair and post-repair) can reduce the time costs, when compared to not adopting the propositions. However, in certain situations, the efficiency appears not to be promoted. In fact, there is a tradeoff between merging equivalent operators and testing the patched versions in terms of time costs. The cases where the optimization does not function can be attributed to a large number of candidate patches (i.e., $|EO|$), thereby much time taken to judge whether the newly chosen estimated operator is the equivalent operator of the previously verified ones. In certain situations, the time for analyzing equivalent operators may exceed that for directly testing the corresponding patched versions. Taking Bugs4Q-id26 for an instance, although the estimated operators can be reduced to half, the time for the case after optimization increases (6.78 s > 4.60 s for pre-repair mode or 6.98 s > 5.06 s for post-repair mode).

Table 3. The Details of Both Cases before and after Applying Propositions to Optimize the Search Space

QPs	$ S_d^R $	$ EO $	$ EO_S $	Pre-repair				Post-repair			
				Before		After		Before		After	
				$ EO_V $	Time	$ EO_V $	Time	$ EO_V $	Time	$ EO_V $	Time
RON-v1	16	8	4	3.92	0.64	1.90	0.40	3.88	0.76	1.96	0.42
RON-v2	4	4	2	4.00	0.52	2.00	0.15	4.00	0.49	2.00	0.25
RON-v3	4	4	2	4.00	0.61	2.00	0.27	4.00	0.35	2.00	0.22
RON-v4	4	4	2	4.00	0.43	2.00	0.27	4.00	0.35	2.00	0.24
RON-v5	16	8	4	4.00	0.60	2.00	0.38	3.66	0.67	1.82	0.39
RCR-v1	65,536	768	384	45.40	62.88	22.74	121.45	0.00	71.14	0.00	217.69
RCR-v2	65,536	768	384	56.30	66.39	31.50	193.89	0.00	57.50	0.00	184.89
RCR-v3	256	64	32	4.02	6.75	1.56	4.34	35.80	17.72	18.48	9.74
RCR-v4	16	16	8	0.00	0.99	0.00	0.64	0.00	1.04	0.00	0.60
RCR-v5	64	32	16	0.00	3.21	0.00	1.73	9.76	7.71	5.18	4.25
CCP-v1	65,536	256	128	0.00	32.38	0.00	58.80	15.10	54.70	8.60	66.60
CCP-v2	16	16	8	0.00	1.58	0.00	0.69	0.00	1.10	0.00	0.88
CCP-v3	256	64	32	1.50	9.57	0.58	6.16	61.52	25.48	30.68	13.67
CCP-v4	64	32	16	0.00	3.25	0.00	1.93	25.12	13.62	12.40	7.14
CCP-v5	16	16	8	0.00	1.57	0.00	0.73	0.00	1.19	0.00	0.82
QFT-v2	65,536	4,096	2,048	4036.10	324.87	1676.40	265.36	1.25	159.49	0.05	41.77
QFT-v3	65,536	4,096	2,048	6.75	54.84	0.10	147.60	99.25	88.51	0.45	148.73
EQ-v1	256	64	32	49.08	16.68	24.36	9.36	26.06	12.59	12.84	7.09
EQ-v2	16	16	8	0.16	1.41	0.08	0.83	0.34	1.39	0.14	1.00
EQ-v3	16	16	8	0.28	1.43	0.14	0.99	0.16	1.35	0.14	0.55
EQ-v4	256	64	32	15.98	7.96	8.00	5.28	13.54	9.12	6.66	6.05
EQ-v5	256	64	32	26.80	15.16	12.92	8.23	9.20	7.81	4.48	4.86
Bugs4Q-id25	256	256	128	256.00	20.21	128.00	10.66	256.00	20.18	128.00	10.61
Bugs4Q-id26	65,536	768	384	58.75	4.60	30.15	6.78	62.05	5.06	31.45	6.98
Bugs4Q-id39	65,536	65,536	1,492	0.00	T/O	0.00	T/O	0.00	T/O	0.00	T/O

The headers “ $|S_d^R|$,” “ $|EO|$,” “ $|EO_S|$,” and “ $|EO_V|$,” respectively, indicate the corresponding sizes of sets mentioned in Section 4. “Time” means how many seconds are taken to traverse all the elements in the search space. The result “T/O” indicates executing the procedure exceeds the time limit of 2h. By repeating the repair procedure to reduce contingency, the data shown below are the average of the 50 samples.

Answer to RQ2: Adopting the propositions for optimization can definitely reduce the number of estimated operators before validating the corresponding patched version. For 76% (19 of 25) of the involved programs, the execution time can be shortened by these propositions.

5.5.3 RQ3: Effect of Patch Correction. To validate the proposed scheme that converts non-unitary operators into unitary ones, this experiment records the error of unitarity ϵ_u and the error of similarity ϵ_s during the procedure of program repair. Consider that for some QPs, a unitary operator cannot be found within an affordable time limit when adopting Uni tAR. Referring to the experimental results in RQ1, the experiment only considers the 28 QPs with candidate unitary patches. For each program, the data shown in the figures are the mean of the errors corresponding to all the possible candidate patches.

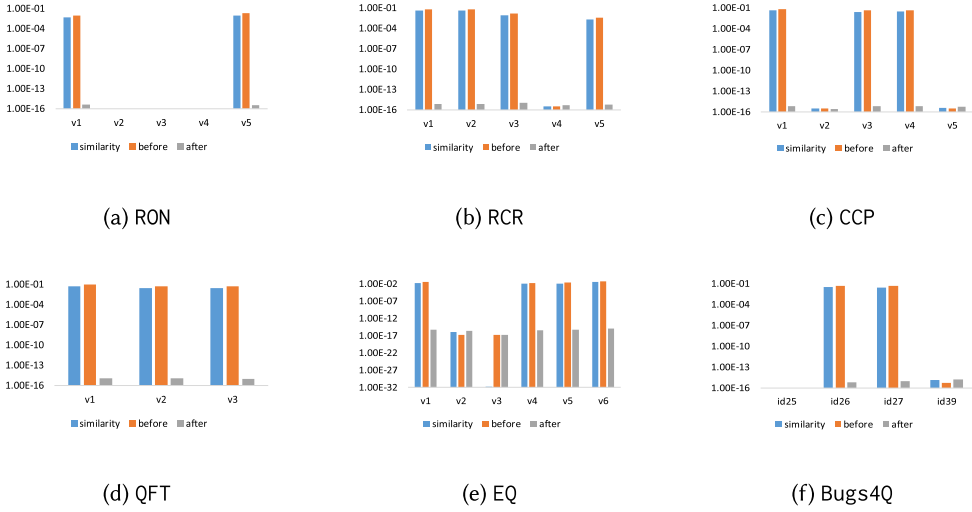


Fig. 5. The bar charts with the logarithmic axis about the errors implicated in the procedure of unitary correction. The legend “similarity” refers to the error of similarity ε_s . “Before” and “after” respectively indicate the errors of unitarity ε_u of candidate patches before and after correction.

As is shown in Figure 5, in 18 of 28 cases (except RON-v2, v3, v4, RCR-v4, CCP-v2, v5, EQ-v2, v3, and Bugs4Q-id25, id39), the errors of unitarity before correction are nearly 10^{-2} , while those after correction can approach 10^{-16} . This illuminates that ε_u can be reduced close to the machine precision of double-precision decimal (decimal64). Hence, the corrected patches can be seen as unitary operators by the relevant API. Although the chosen method, i.e., Householder reflections, to implement the QR decomposition is numerically stable, the procedure of numerical computation inevitably introduces additional tiny errors. In view of that, most of the ε_u after correction is not strictly equal to 0. Exceptionally, for RON-v2, v3, v4, and Bugs4Q-id25, the corresponding ε_u after correction even equal 0 rather than approaching that, which hints that no more errors are introduced during the QR decomposition. The above two cases may be credited to the mathematically particular forms of patches.

As to the error of similarity ε_s , the heights of corresponding bars whose legend is “similarity” are about either 10^{-2} or 10^{-16} . These values are numerically close to respective ε_u before correction. The result can demonstrate that the correction converts a patch to a unitary one by subtle transformation, so the proposed unitary correction can guarantee not to yield an unexpected unitary operator.

For the purpose of further analyzing the effect of patch correction, a case study is conducted to investigate the two special patches involved in the procedure of program repair. Shown in Table 4, the error of unitarity ε_u of one patch stays 0, while that of the other patch increases from 0. Each row or column of the first patch has only one non-zero element, and this element is either ± 1 or $\pm 1i$. Then, the numerical computation to implement the QR decomposition will not produce the floating point errors. For the second operator, each element possesses the same absolute value (i.e., 0.5), and this patch is a strictly unitary operator before correction. Nevertheless, when performing numerical computations, additional errors may occur, such as the preceding 0 or $0i$ being replaced by the small values. This results in a corrected patch with a tiny ε_u . Therefore, the impact of errors brought by numerical computations should be taken into account, when using classical computers to execute QPs.

Table 4. The Details of Two Special Patches Processed by Unitary Correction Where the Data Displayed Are Accurate to Three Significant Digits

Descriptions	Correction	Patches	ϵ_u	ϵ_s
ϵ_u stays 0.	Before	$\begin{bmatrix} 0+0i & 0+0i & 0-1i & 0+0i \\ 1+0i & 0+0i & 0+0i & 0+0i \\ 0+0i & 0+1i & 0+0i & 0+0i \\ 0+0i & 0+0i & 0+0i & -1+0i \end{bmatrix}$	0.00E+00	0.00E+00
	After	$\begin{bmatrix} 0+0i & 0+0i & 0-1i & 0+0i \\ 1+0i & 0+0i & 0+0i & 0+0i \\ 0+0i & 0+1i & 0+0i & 0+0i \\ 0+0i & 0+0i & 0+0i & -1+0i \end{bmatrix}$	0.00E+00	
ϵ_u increases from 0.	Before	$\begin{bmatrix} -0.5+0i & 0+0.5i & 0.5+0i & 0-0.5i \\ -0.5+0i & 0-0.5i & 0.5+0i & 0+0.5i \\ -0.5+0i & 0.5+0i & -0.5+0i & 0.5+0i \\ -0.5+0i & -0.5+0i & -0.5+0i & -0.5+0i \end{bmatrix}$	0.00E+00	3.55E-16
	After	$\begin{bmatrix} -0.5+0i & 0+0.5i & 0.5+3.82 \times 10^{-17}i & 1.67 \times 10^{-16} - 0.5i \\ -0.5+0i & 0-0.5i & 0.5-7.05 \times 10^{-17}i & 5.55 \times 10^{-17} + 0.5i \\ -0.5+0i & 0.5-2.78 \times 10^{-17}i & -0.5-2.20 \times 10^{-17}i & 0.5+8.33 \times 10^{-17}i \\ -0.5+0i & -0.5-2.78 \times 10^{-17}i & -0.5+5.44 \times 10^{-17}i & -0.5+5.55 \times 10^{-17}i \end{bmatrix}$	5.93E-16	

Answer to RQ3: For the discussed 28 programs, unitary correction can effectively control all the errors of unitarity to within the order of magnitudes 10^{-16} , where 18 programs are bothered with the high errors of unitarity (i.e., around 10^{-1} or 10^{-2}) before correction. Meanwhile, unitary correction produces desired patches subtly different from the original ones, where the corresponding errors of similarity are about 10^{-16} .

5.6 Threats to Validity

- *Internal validity:* The threats to internal validity include specific algebraic operations and test oracle problems. QPs are sensitive to the algebraic operation implemented by quantum gates, so different defects may cause diverse repair results. For example, the three mentioned APR methods cannot fix CCP-v2 with two defects but can repair CCP-v3 with the same number of defects. To cover as many cases as possible, the buggy QPs selected in the article are equipped with various numbers and diverse kinds of defects. With regard to the existing measurement-based test oracle, it becomes invalid in distinguishing the phases of a final state. That is why the buggy QP Bugs4Q-id3 is identified as a bug-free one by mistake. Moreover, the oracle problems result in distinct validation results of the same patched version. As is shown in RQ2, the mean values of $|EO_V|$ are not always integers. Hence, this issue remains to be investigated and solved in future work.
- *External validity:* Due to the limited numbers of available QPs and relevant databases, the chosen buggy programs cannot represent all the possible QPs, especially the ones with numerous qubits and complicated structures. For the purpose of mitigating the above threats, the article employs 34 buggy versions (29 are generated by mutation, and 5 are collected from the real world) with distinct numbers of qubits and depths of quantum circuits. Although current developers tend to code QPs with little scale, when the quantum projects become more complex and sophisticated in the future, more comprehensive studies should be conducted to refine the APR methods.
- *Construct validity:* The construct threats may relate to dependent variables and the metric for oracle. The mentioned dependent variables are simple in concept and straightforward to apply, and they contribute to assessing the effectiveness and efficiency of the repair

procedure. As for the metric to examine whether the test passes or fails, previous studies [3, 41, 55] have assessed the statistical significance of the results, such as Wilcoxon signed-rank test and Pearson's chi-square test. Strictly, the effectiveness of statistical hypothesis tests depends on the particular probability distribution and the size of samples, whereas the probabilistic output of a QP can be an arbitrary discrete distribution. In machine learning, distance metrics are broadly applied to quantify the divergence between two probability distributions. Therefore, to avoid the threat brought by possible unreasonable statistical tests, the article just employs a kind of distance metric (J-S divergence) and a variable threshold, where this approach can play an analogous role to the statistical tests and even have no limitation to the distributions of data.

- *Conclusion validity*: Due to the randomness involved in the measurement of QPs and the fluctuation of dependent variables, it is difficult for researchers to replicate the identical results displayed in experiments. However, enough shots and unitary correction can limit the negative impact of randomness from quantum measurement. For the dependent variables, such as the execution time and the errors, RQ1 configures a period of 2h for the investigation of repairing results. RQ2 and RQ3 use mean values to obtain statistically significant data from multiple trials. Therefore, the slight deviation in observations hardly poses a significant threat to conclusion validity.

6 CONCLUSIONS AND FUTURE WORK

With the characteristics of superposition and entanglement in QC, QPs can implement computation and simulation in exceedingly complicated tasks. At the same time, defects during qubit evolution, which induce program failures with incorrect functionality, pose a severe threat to the quality of QPs. Thus, an accessible QC-oriented method of program repair is intensively required so that an autonomous, dependable and affordable quantum software system can be realized. This article proposes a new method, namely UniTar, to cope with the automatic repair issue of QPs. Then, empirical studies are also conducted to verify the practicability of UniTar.

The theoretical method UniTar incorporates the modeling of a QP, the process of patch generation and validation, as well as unitary correction. Both the expected and defective QPs can be abstracted as unitary operators, so a suitable patch designed to perform an additional unitary operator corrects the latter program into the former one. The abundant test results play a direct part in analyzing the black box and generating available patches. For the iterative phase of patches, two propositions can improve the efficiency in accessing operators and validating patches, respectively by shrinking the search space and merging the equivalent operators. As an indispensable step to ensure the smooth operation of patch addition, unitary correction can effectively transfer a non-unitary operator into an acceptably unitary one, where the two operators have adjacent norms.

The experiments are designed to evaluate UniTar in terms of repair results, optimization for searching, and effect of unitary correction. According to the repair results of defective versions, UniTar can fix more programs than both baselines (GenProg and TBar). Besides, in comparison to the two baselines, UniTar can generally generate more patches that fix the programs and also consume less time to find such appropriate patches. As for the generate-and-validate procedure, the propositions for optimization can reduce the size of the search space by merging equivalent patches. In most cases, the execution time can be shortened as well. The procedure of unitary correction can significantly eliminate the negative impacts from the randomness of quantum measurement and float-point errors in classical numerical computing, such that the candidate patches can be ensured as expected unitary operators.

In future work, more attention should be paid to lowering the number of test suites and speeding up searching patches. The patches are defined as large unitary matrices, thereby confining

the efficiency of program repair when the number of qubits increases. An optimized model or technique utilizing fewer test cases to estimate the black box can be considered later. Besides, in analogy to fault detection in software testing, the part of patch generation and validation naturally forms a feedback-based loop. According to the principle of software cybernetics [4, 7, 61], the patch searching can be regarded as a control process, where previous results of validation guide the next choice of alternative patches to find out fixed versions more quickly.

APPENDICES

A RELATED LEMMAS AND PROPOSITIONS

LEMMA A.1 (COMPLETENESS RELATION). *Let $|\varphi^{(x)}\rangle$ be any orthonormal basis for the vector space $\mathcal{H}$, so an arbitrary vector $|v\rangle$ can be written $|v\rangle = \sum_x v_x |\varphi^{(x)}\rangle$ for some set of complex numbers v_i . Then, all the $|v\rangle$ follow that $\sum_x |\varphi^{(x)}\rangle \langle \varphi^{(x)}| = I$.*

PROOF. This is a lemma widely used in quantum information theory. The detail of proof can be found in the literature [43]. $\square$

LEMMA A.2 (PROPERTY OF NORM). *Let $A \in \mathbb{C}^{2^n \times 2^n}$ and $\|A\|$ be some matrix norm. If $\|A\| < 1$, then $I \pm A$ is nonsingular.*

PROOF. This is a lemma utilized to connect the matrix norm with invertibility. The proof's details can be found in most literature or textbooks relevant to linear algebra and matrix theory. $\square$

LEMMA A.3 (UNIQUENESS OF QR DECOMPOSITION). *Given a 2^n -order complex matrix A , it can be decomposed into $A = QR$, in which Q and R are respectively denoted as a unitary matrix and an upper triangular matrix with nonnegative diagonal elements. In the case where A is invertible, the decomposition is identified as unique.*

PROOF. This lemma illuminates how a nonsingular matrix works when performing QR decomposition. The proof's details can be also found in most literature or textbooks relevant to linear algebra and matrix theory. $\square$

PROPOSITION A.1. *For any diagonal unitary operator $D_d^{(x)}$ and final state $|\psi_d^{(x)}\rangle \in \mathcal{H}_d$, $D^{(x)}|\psi_d^{(x)}\rangle$ has the same measurement results as $|\psi_d^{(x)}\rangle$.*

PROOF. See Appendix B.1. $\square$

PROPOSITION A.2. *The sign operator S possesses the following properties.*

- (I) S is a non-linear operator.
- (II) When $a, b \in \mathbb{R}$ and $|a| > |b|$, $S(a + b) = S(a)$.
- (III) As $a \in \mathbb{C}$, $\text{Re}(S(a)) = S(\text{Re}(a))$.

PROOF. See Appendix B.2. $\square$

PROPOSITION A.3. *The patch $U_p^{(z)}$ defined by (31) is determined to be full rank.*

PROOF. See Appendix B.3. $\square$

B PROOFS OF PROPOSITIONS

B.1 Proof of Proposition A.1

PROOF. Let $D_d^{(x)}$ be composed of phase elements, i.e.,

$$D_d^{(x)} = \text{diag} \left\{ e^{i\theta_{x,0}}, e^{i\theta_{x,1}}, \dots, e^{i\theta_{x,2^n-1}} \right\},$$

where $\theta_{x,0}, \theta_{x,1}, \dots, \theta_{x,2^n-1} \in [0, 2\pi)$. And

$$\left(D_d^{(x)}|\psi_d^{(x)}\rangle\right)^\dagger = \langle\psi_d^{(x)}| \left(D_d^{(x)}\right)^\dagger = \langle\psi_d^{(x)}|\text{diag}\left\{e^{i\theta_{x,0}}, e^{i\theta_{x,1}}, \dots, e^{i\theta_{x,2^n-1}}\right\}.$$

Then introduce the collection of projectors $\{M_y\}_{y=0}^{2^n-1}$, where specifically,

$$M_y = |y\rangle\langle y| = \text{diag}\{\underbrace{0, 0, \dots, 0}_y, \underbrace{1, 0, \dots, 0}_{2^n-y-1}\}.$$

For the probability of the result y based on the input x ,

$$p_{Y|X}(y|x) = \langle\psi_d^{(x)}| \left(D_d^{(x)}\right)^\dagger M_y D_d^{(x)} |\psi_d^{(x)}\rangle,$$

where

$$\left(D_d^{(x)}\right)^\dagger M_y D_d^{(x)} = \text{diag}\{\underbrace{0, 0, \dots, 0}_y, \underbrace{e^{-i\theta_{x,y}} e^{i\theta_{x,y}}}_{2^n-y-1}, \dots, 0\} = M_y.$$

Consequently,

$$\forall y \in \{0, 1, \dots, 2^n - 1\} : \langle\psi_d^{(x)}| \left(D_d^{(x)}\right)^\dagger M_y D_d^{(x)} |\psi_d^{(x)}\rangle = \langle\psi_d^{(x)}| M_y |\psi_d^{(x)}\rangle,$$

which infers the identical measurement results of $D_d^{(x)}|\psi_d^{(x)}\rangle$ and $|\psi_d^{(x)}\rangle$. $\square$

B.2 Proof of Proposition A.2

PROOF. (I) A linear operator f satisfies homogeneity that $f(\beta x) = \beta f(x)$ for all possible β and additivity that $f(x + y) = f(x) + f(y)$ where x and y are independent variables. By that means, take a counterexample that given $\beta = 2$ and $x = -1$, $S(\beta x) = S(2 \times (-1)) = -1$ while $\beta S(x) = 2 \times S(-1) = -2$, which contradicts with homogeneity for $S(\beta x) \neq \beta S(x)$. Thus, S turns out to be a non-linear operator.

(II) In one case that $S(a) = S(b)$, $a + b = S(a) \cdot (|a| + |b|)$ where $|a| + |b| > 0$, and then $S(a + b) = S(a)$. In the other case that $S(a) \neq S(b)$, $a + b = S(a) \cdot (|a| - |b|)$ where $|a| - |b| > 0$, and then $S(a + b) = S(a)$.

(III) Supposed $a = \text{Re}(a) + \text{Im}(a)i$, $\text{Re}(S(a)) = \text{Re}(S(\text{Re}(a)) + S(\text{Im}(a))i) = \text{Re}(S(\text{Re}(a))) = S(\text{Re}(a))$, in which $S(\text{Re}(a)) \in \mathbb{R}$. $\square$

B.3 Proof of Proposition A.3

PROOF. Since $\bar{U}_p^{(z)}$ is a unitary matrix, its norm satisfies $\|\bar{U}_p^{(z)}\|_2 = 1$, in which the following proof process only discusses the two-norm and omits the subscript. Because of E_p , an operator of tiny disturbance, it can be considered that each element in E is tiny enough. Then,

$$\|E_p\| < \|\bar{U}_p^{(z)}\| = \left\| \left(\bar{U}_p^{(z)}\right)^{-1} \right\| = 1.$$

Then,

$$\left\| \left(\bar{U}_p^{(z)}\right)^{-1} E_p \right\| \leq \left\| \left(\bar{U}_p^{(z)}\right)^{-1} \right\| \cdot \|E_p\| = \|E_p\| < 1.$$

Owing to

$$U_p^{(z)} \equiv \bar{U}_p^{(z)} + E_p = \bar{U}_p^{(z)} \left(I + \left(\bar{U}_p^{(z)}\right)^{-1} E_p \right),$$

and Lemma A.2, $I + (\bar{U}_p^{(z)})^{-1} E_p$ and then $\bar{U}_p^{(z)} + E_p$ are non-singular, i.e., the matrix $U_p^{(z)}$ with a small disturbance is still of full rank. $\square$

B.4 Proof of Proposition 1

PROOF. (I) Given $k_E = 2^n$, $\hat{U}$ can be expressed as

$$\hat{U} = \sum_{x=0}^{2^n-1} |\psi^{(x)}\rangle \langle \varphi^{(x)}| = \sum_{x=0}^{2^n-1} U |\varphi_x\rangle \langle \varphi^{(x)}| = U\Phi.$$

According to Lemma A.1, the relation $\hat{U} = U$ holds, which identifies $\hat{U}$ as a unitary operator.

(II) In the virtue of eigendecomposition, a unitary matrix C_φ and a diagonal matrix D_φ are introduced as

$$\Phi = C_\varphi D_\varphi C_\varphi^{-1},$$

where to be specific,

$$C_\varphi = \left[|\varphi^{(0)}\rangle, |\varphi^{(1)}\rangle, \dots, |\varphi^{(2^n-1)}\rangle \right],$$

$$D_\varphi = \text{diag} \left\{ \overbrace{1, 1, \dots, 1}^{k_E}, \overbrace{0, \dots, 0}^{2^n-k_E} \right\}.$$

For Φ , $|\varphi_x\rangle$ ($x = 0, 1, \dots, k_E - 1$) is the eigenvector with the eigenvalue 1. And it is possible to construct the collection of orthogonal basis states $\{|\varphi_x\rangle\}_{x=k_E}^{2^n-1}$ in the complex vector space $\mathcal{H}_i$ with $\dim(\mathcal{H}_i) = 2^n$. Then it is clear that $\text{rank}(D_\varphi) = k_E$. Based on the linear transformation by full-rank matrices U, C_φ and C_φ^{-1} ,

$$\text{rank}(\hat{U}) = \text{rank}(U\Phi) = \text{rank}(UC_\varphi D_\varphi C_\varphi^{-1}) = \text{rank}(D_\varphi) = k_E. \quad \square$$

B.5 Proof of Proposition 2

PROOF. Upon the *contradiction*, first assume the existence of linear dependence that

$$\exists m \neq m' : |s^{(m)}\rangle = h \cdot |s^{(m')}\rangle, h \in \mathbb{R}, \quad (\text{b.1})$$

where $|s^{(m)}\rangle$ is short for $S|u^{(m)}\rangle$.

Then, let

$$\begin{aligned} |u^{(m)}\rangle &\equiv [u_{1,m} + u_{1,m}^* i, u_{2,m} + u_{2,m}^* i, \dots]^\dagger = [s_{1,m} |u_{1,m}| + s_{1,m}^* |u_{1,m}^*| i, s_{2,m} |u_{2,m}| + s_{2,m}^* |u_{2,m}^*| i, \dots]^\dagger, \\ |s^{(m)}\rangle &\equiv [s_{1,m} + s_{1,m}^* i, s_{2,m} + s_{2,m}^* i, \dots]^\dagger, \end{aligned}$$

where $u_{1,m}, u_{2,m}, \dots \in \mathbb{R}, u_{1,m}^*, u_{2,m}^*, \dots \in \mathbb{R}$ and $s_{1,m}, s_{2,m}, \dots \in \{-1, 1\}, s_{1,m}^*, s_{2,m}^*, \dots \in \{-1, 1\}$. According to Equation (b.1),

$$\forall l \in \mathbb{N} : s_{l,m} = h s_{l,m'}, s_{l,m}^* = h s_{l,m'}^*.$$

For the absolute value of the inner product of $|u^{(m)}\rangle$ and $|u^{(m')}\rangle$, it can be concluded that $|\langle u^{(m)} | u^{(m')} \rangle| \geq 0$ through

$$\begin{aligned} |\langle u^{(m)} | u^{(m')} \rangle| &= \left| \sum_{l=1}^{\infty} (s_{l,m} |u_{l,m}| - s_{l,m}^* |u_{l,m}^*| i) (s_{l,m'} |u_{l,m'}| + s_{l,m'}^* |u_{l,m'}^*| i) \right| \\ &= \left| \sum_{l=1}^{\infty} h (s_{l,m'} |u_{l,m'}| - s_{l,m'}^* |u_{l,m'}^*| i) (s_{l,m'} |u_{l,m'}| + s_{l,m'}^* |u_{l,m'}^*| i) \right| \\ &= |h| \left| \sum_{l=1}^{\infty} [s_{l,m'}^2 |u_{l,m'} u_{l,m'}| + (s_{l,m'}^*)^2 |u_{l,m'}^* u_{l,m'}^*|] + s_{l,m'} s_{l,m'}^* (|u_{l,m'} u_{l,m'}^*| - |u_{l,m'}^* u_{l,m'}|) i \right| \end{aligned}$$

$$\begin{aligned}
&\geq |h| \left| \sum_{l=1}^{\infty} |u_{l,m} u_{l,m'}| + |u_{l,m}^* u_{l,m'}^*| \right| \\
&\geq |h| \sum_{l=1}^{\infty} |u_{l,m} u_{l,m'} + u_{l,m}^* u_{l,m'}^*| \\
&\geq 0.
\end{aligned} \tag{b.2}$$

In detail, it is important to determine if the inner product can equal zero. Clearly, this case holds only if $\forall l \in \mathbb{N}, \exists m, m' \in \mathbb{N} : |u_{l,m} u_{l,m'}| + |u_{l,m}^* u_{l,m'}^*| = 0$ and $|u_{l,m} u_{l,m'}^*| = |u_{l,m}^* u_{l,m'}|$, which implicates $|u^{(m)}\rangle = |u^{(m')}\rangle = 0$. Therefore, it can be concluded $\text{rank}(\{|u^{(m)}\rangle\}_{m=0}^{2^n-1}) < 2^n$ and $\{|u^{(m)}\rangle\}_{m=0}^{2^n-1}$ as a non-orthogonal basis state collection. Because of $|\langle u^{(m)} | u^{(m')} \rangle| > 0$ in contradiction to the orthogonality $\langle u^{(m)} | u^{(m')} \rangle = 0$, any two different column vectors $|s^{(m)}\rangle, |s^{(m')}\rangle$ are linearly independent. $\square$

B.6 Proof of Proposition 3

PROOF. The size of $|\mathcal{S}_d^{\mathbb{R}}|$ reaches its maximum when *there is no 0 in the $2^n \times 2^n$ real matrix $\hat{U}_{d0}$* , because, for the value of each element, there are 2 possibilities by multiplying -1 or $+1$, while the multiplication of 0 decreases to the sole possible value 0. Without the guidance from Proposition 2, there are $2^{2^n \times 2^n} = 2^{4^n}$ alternative states to be searched, which shows that $\max_{\hat{U}_{d0}} |\mathcal{S}_d^{\mathbb{R}}| = 2^{4^n}$ in the former situation.

In comparison, when considering the proposition about linear independence after the operation by the sign operator, the phase modulation to elements by signs $+1$ and -1 follows that each column can not be assigned with the same or opposite signs. Due to 2^n columns, 2^{2^n} cases can be constructed in different sign allocations. Analogous to that n bits generate 2^n numbers, their corresponding binary numbers represent specific sign assignments, and for example, it denotes signs of one column as $[-1, -1, +1, -1]^T$ when $2 = (0010)_b$ is chosen. The second constraint that any sign allocations of two columns are not opposite can be considered by symmetry. To be explicit, when choosing 2^{2^n} distinct kinds of sign allocations, there must be 2^{2^n-1} pairs of columns with opposite signs, such pairs as $[+1, +1, +1, -1]^T$ and $[-1, -1, -1, +1]^T$ with $n = 2$. Thus, it can be abstracted as the steps sequentially selecting 2^n different pairs from 2^{2^n-1} choices. Given the pairs, the account of total cases are

$$A_{2^{2^n-1}}^{2^n} = \frac{2^{2^n-1}!}{(2^{2^n-1} - 2^n)!},$$

where the number of permutation $A_N^M (M, N \in \mathbb{N}^*, M \leq N)$ is defined as $N!/(N-M)!$. Because the above pairs are constructed in 2^{2^n} possibilities, the maximum size of the search space can be calculated as

$$\max_{\hat{U}_{d0}} |\mathcal{S}_d^{\mathbb{R}}| = 2^{2^n} A_{2^{2^n-1}}^{2^n} = \frac{2^{2^n} \cdot (2^{2^n-1})!}{(2^{2^n-1} - 2^n)!}. \quad \square$$

B.7 Proof of Proposition 4

PROOF. To prove completely, two repair modes, pre-repair and post-repair, must be discussed as follows.

For *Pre-repair*,

$$U_r^{(z)} = U_d U_{ep}^{(z)} = \left(\Delta_d^{(z)} + I \right)^{-1} \hat{U}_d^{(z)} \left(\hat{U}_d^{(z)} \right)^{-1} U_e = \left(\Delta_d^{(z)} + I \right)^{-1} U_e.$$

Because $(\Delta_d^{(z)} + I)$ is confirmed as a unitary operator, the probability of measurement satisfies

$$\begin{aligned} p_{Y_R|X,Z}(y|x,z) &= \langle \varphi^{(x)} | \left(U_r^{(z)} \right)^\dagger P_y U_r^{(z)} | \varphi^{(x)} \rangle \\ &= \langle \varphi^{(x)} | U_e^\dagger \left(\Delta_d^{(z)} + I \right) P_y \left(\Delta_d^{(z)} + I \right)^{-1} U_e | \varphi^{(x)} \rangle. \end{aligned} \quad (\text{b.3})$$

Providing that, for $\theta \in [0, 2\pi)$,

$$U_\Delta^{(z,z')} = e^{i\theta} I = e^{i\theta} \left(\Delta_d^{(z)} + I \right)^{-1} \left(\Delta_d^{(z')} + I \right) = \left(\Delta_d^{(z')} + I \right)^{-1} \left(\Delta_d^{(z)} + I \right),$$

then

$$\left(\Delta_d^{(z')} + I \right) = e^{-i\theta} \left(\Delta_d^{(z)} + I \right).$$

Further,

$$\left(\Delta_d^{(z)} + I \right)^{-1} \left(\Delta_d^{(z')} + I \right) = e^{-i\theta} I,$$

which states that $(\Delta_d^{(z)} + I)^{-1}(\Delta_d^{(z')} + I)$ is a scalar matrix as well. Because of the commutative property when multiplying a scalar matrix, i.e., $[M_y, (\Delta_d^{(z)} + I)^{-1}(\Delta_d^{(z')} + I)] = 0$ and

$$M_y \left(\Delta_d^{(z)} + I \right)^{-1} \left(\Delta_d^{(z')} + I \right) = \left(\Delta_d^{(z)} + I \right)^{-1} \left(\Delta_d^{(z')} + I \right) M_y.$$

Then,

$$\left(\Delta_d^{(z)} + I \right) M_y \left(\Delta_d^{(z')} + I \right)^{-1} = \left(\Delta_d^{(z')} + I \right) M_y \left(\Delta_d^{(z')} + I \right)^{-1},$$

that stands for the equation $\forall y, \forall x : p_{Y_R|X,Z}(y|x,z) = p_{Y_R|X,Z}(y|x,z')$ and proves $\hat{U}_d^{(z)}$ and $\hat{U}_d^{(z')}$ are equivalent.

For *Post-repair*,

$$U_r^{(z)} = U_{op}^{(z)} U_d = U_e \left(\hat{U}_d^{(z)} \right)^{-1} \left(\Delta_d^{(z)} + I \right) \hat{U}_d^{(z)}.$$

Then, the probability of measurement satisfies

$$\begin{aligned} p_{Y_R|X,Z}(y|x,z) &= \langle \varphi^{(x)} | \left(U_r^{(z)} \right)^\dagger P_y U_r^{(z)} | \varphi^{(x)} \rangle \\ &= \langle \varphi^{(x)} | \left(\hat{U}_d^{(z)} \right)^\dagger \left(\Delta_d^{(z)} + I \right)^{-1} \hat{U}_d^{(z)} U_e^\dagger P_y U_e \left(\hat{U}_d^{(z)} \right)^\dagger \left(\Delta_d^{(z)} + I \right) \hat{U}_d^{(z)} | \varphi^{(x)} \rangle \\ &= \langle \varphi^{(x)} | \left(A^{(z)} \right)^\dagger B^{(y)} A^{(z)} | \varphi^{(x)} \rangle. \end{aligned} \quad (\text{b.4})$$

in which some of the operators can be abbreviated as $A^{(z)} = (\hat{U}_d^{(z)})^\dagger U_d$, $B^{(y)} = U_e^\dagger P_y U_e$.

Supposed for $\theta \in [0, 2\pi)$,

$$U_\Delta^{(z,z')} = e^{i\theta} I = e^{i\theta} \hat{U}_d^{(z)} \left(\hat{U}_d^{(z')} \right)^\dagger = \hat{U}_d^{(z)} \left(\hat{U}_d^{(z')} \right)^\dagger.$$

Then,

$$A^{(z)} \left(A^{(z')} \right)^\dagger = \hat{U}_d^{(z)} U_d U_d^\dagger \left(\hat{U}_d^{(z')} \right)^\dagger = \hat{U}_d^{(z)} \left(\hat{U}_d^{(z')} \right)^\dagger = U_\Delta^{(z,z')}.$$

Due to the commutative property when multiplying a scalar matrix, i.e., $[B^{(y)}, A^{(z)}(A^{(z')})^\dagger] = 0$ and

$$B^{(y)} A^{(z)} \left(A^{(z')} \right)^\dagger = A^{(z)} \left(A^{(z')} \right)^\dagger B^{(y)}.$$

Then,

$$\left(A^{(z)} \right)^\dagger B^{(y)} A^{(z)} = \left(A^{(z')} \right)^\dagger B^{(y)} A^{(z')},$$

that also indicates the equation $\forall x, \forall y : p_{Y_R|X,Z}(y|x, z) = p_{Y_R|X,Z}(y|x, z')$ and proves $\hat{U}_d^{(z)}$ and $\hat{U}_d^{(z')}$ to be *equivalent*. $\square$

B.8 Proof of Proposition 5

PROOF. According to Proposition A.3, $U_p^{(z)}$ is identified as a non-singular operator, which denotes that there exists a unique QR decomposition in the condition of Lemma A.3. Assuming that $R_p = (r_{m,m'})_{2^n \times 2^n}$ is denoted as an upper triangular matrix with real diagonal values. Due to R_p of full rank, $\det(R_p) = \prod_m r_{m,m} > 0$, showing that none of the diagonal values equals to 0.

Let operators be represented by column vectors that $U_p = [|u_p^{(1)}\rangle, |u_p^{(2)}\rangle, \dots, |u_p^{(2^n)}\rangle]$, $\bar{U}_p = [|\bar{u}_p^{(1)}\rangle, |\bar{u}_p^{(2)}\rangle, \dots, |\bar{u}_p^{(2^n)}\rangle]$ and $\tilde{U}_p = [|\tilde{u}_p^{(1)}\rangle, |\tilde{u}_p^{(2)}\rangle, \dots, |\tilde{u}_p^{(2^n)}\rangle]$. Then, propositions (I) and (II) are proved by discussing the following two cases.

In the first case that $\forall m : r_{m,m} > 0$, the Gram–Schmidt process can specify QR decomposition.

(I) Then, each column vector can be expressed as follows:

- If $m = 1$, then

$$\begin{aligned} |v_p^{(1)}\rangle &= |u_p^{(1)}\rangle, \\ |\tilde{u}_p^{(1)}\rangle &= \frac{|v_p^{(1)}\rangle}{\sqrt{\langle v_p^{(1)} | v_p^{(1)} \rangle}}. \end{aligned} \quad (\text{b.5})$$

- If $1 < m \leq 2^n$, then

$$\begin{aligned} |v_p^{(m)}\rangle &= |u_p^{(m)}\rangle - \sum_{l=1}^{m-1} \langle \tilde{u}_p^{(l)} | u_p^{(m)} \rangle |\tilde{u}_p^{(l)}\rangle, \\ |\tilde{u}_p^{(m)}\rangle &= \frac{|v_p^{(m)}\rangle}{\sqrt{\langle v_p^{(m)} | v_p^{(m)} \rangle}}. \end{aligned} \quad (\text{b.6})$$

It is worth noting that E_p with small values, i.e., $E_p \rightarrow 0$ and $\forall m = 1, 2, \dots, 2^n : |e_p^{(m)}\rangle \rightarrow 0 + j0$. Then, the proof that $\forall m : \lim_{E_p \rightarrow 0} |\tilde{u}_p^{(m)}\rangle = |\bar{u}_p^{(m)}\rangle$ is conducted by the mathematical induction.

- For $m = 1$,

$$\begin{aligned} \lim_{E_p \rightarrow 0} |\tilde{u}_p^{(1)}\rangle &= \lim_{|e_p^{(1)}\rangle \rightarrow 0} \frac{|v_p^{(1)}\rangle}{\sqrt{\langle v_p^{(1)} | v_p^{(1)} \rangle}} \\ &= \lim_{|e_p^{(1)}\rangle \rightarrow 0} \frac{|\bar{u}_p^{(1)}\rangle + |e_p^{(1)}\rangle}{\sqrt{\left(\langle \bar{u}_p^{(1)} | + \langle e_p^{(1)} | \right) \left(|\bar{u}_p^{(1)}\rangle + |e_p^{(1)}\rangle \right)}} \\ &= \lim_{|e_p^{(1)}\rangle \rightarrow 0} \frac{|\bar{u}_p^{(1)}\rangle + |e_p^{(1)}\rangle}{\sqrt{1 + \langle \bar{u}_p^{(1)} | e_p^{(1)} \rangle + \langle e_p^{(1)} | \bar{u}_p^{(1)} \rangle + \langle e_p^{(1)} | e_p^{(1)} \rangle}} \\ &= |\bar{u}_p^{(1)}\rangle. \end{aligned} \quad (\text{b.7})$$

- If for $m = l$ and $\lim_{E_p \rightarrow 0} |\tilde{u}_p^{(m)}\rangle = |\bar{u}_p^{(m)}\rangle$ ($m = 1, 2, \dots, l; l < 2^n$) holds, then for $m = l + 1$,

$$\begin{aligned}
\lim_{E_p \rightarrow 0} |\tilde{v}_p^{(l+1)}\rangle &= \lim_{E_p \rightarrow 0} \left(|u_p^{(l+1)}\rangle - \sum_{m=1}^l \langle \tilde{u}_p^{(m)} | u_p^{(l+1)} \rangle |\tilde{u}_p^{(m)}\rangle \right) \\
&= \lim_{E_p \rightarrow 0} \left[\left(|\bar{u}_p^{(l+1)}\rangle + |e_p^{(l+1)}\rangle \right) - \sum_{m=1}^l \langle \tilde{u}_p^{(m)} | \left(|\bar{u}_p^{(l+1)}\rangle + |e_p^{(l+1)}\rangle \right) |\tilde{u}_p^{(m)}\rangle \right] \\
&= \lim_{E_p \rightarrow 0} \left(|\bar{u}_p^{(l+1)}\rangle - \sum_{m=1}^l \langle \tilde{u}_p^{(m)} | \bar{u}_p^{(l+1)} \rangle |\tilde{u}_p^{(m)}\rangle \right) \\
&= |\bar{u}_p^{(l+1)}\rangle - \sum_{m=1}^l \langle \bar{u}_p^{(m)} | \bar{u}_p^{(l+1)} \rangle |\bar{u}_p^{(m)}\rangle \\
&= |\bar{u}_p^{(l+1)}\rangle.
\end{aligned} \tag{b.8}$$

Hence,

$$\lim_{E_p \rightarrow 0} |\tilde{u}_p^{(l+1)}\rangle = \lim_{E_p \rightarrow 0} \frac{|\tilde{v}_p^{(l+1)}\rangle}{\sqrt{\langle \tilde{v}_p^{(l+1)} | \tilde{v}_p^{(l+1)} \rangle}} = \frac{|\bar{u}_p^{(l+1)}\rangle}{\sqrt{\langle \bar{u}_p^{(l+1)} | \bar{u}_p^{(l+1)} \rangle}} = |\bar{u}_p^{(l+1)}\rangle. \tag{b.9}$$

Generally, $\lim_{E_p \rightarrow 0} |\tilde{u}_p^{(m)}\rangle = |\bar{u}_p^{(m)}\rangle$, $m = 1, 2, \dots, 2^n$ sets up. As it is obvious that $\lim_{E_p \rightarrow 0} U_p = \bar{U}_p$, it can be shown that $\lim_{E_p \rightarrow 0} \bar{U}_p - U_p = O$.

(II) With regard to $R_p = R_p^{(z)}$ decomposed from $U_p^{(z)}$, it is denoted in the form of inner product,

$$R_p = \begin{bmatrix} \sqrt{\langle v_p^{(1)} | v_p^{(1)} \rangle} & \langle \tilde{u}_p^{(1)} | u_p^{(2)} \rangle & \cdots & \langle \tilde{u}_p^{(1)} | u_p^{(2^n-1)} \rangle & \langle \tilde{u}_p^{(1)} | u_p^{(2^n)} \rangle \\ 0 & \sqrt{\langle v_p^{(2)} | v_p^{(2)} \rangle} & \cdots & \langle \tilde{u}_p^{(2)} | u_p^{(2^n-1)} \rangle & \langle \tilde{u}_p^{(2)} | u_p^{(2^n)} \rangle \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & \sqrt{\langle v_p^{(2^n-1)} | v_p^{(2^n-1)} \rangle} & \langle \tilde{u}_p^{(2^n-1)} | u_p^{(2^n)} \rangle \\ 0 & 0 & \cdots & 0 & \sqrt{\langle v_p^{(2^n)} | v_p^{(2^n)} \rangle} \end{bmatrix},$$

where based on Equations (b.7), (b.8), and (b.9),

$$\begin{aligned}
\lim_{E_p \rightarrow 0} \sqrt{\langle v_p^{(m)} | v_p^{(m)} \rangle} &= \sqrt{\langle \bar{u}_p^{(m)} | \bar{u}_p^{(m)} \rangle} = 1, \\
\lim_{E_p \rightarrow 0} \langle \tilde{u}_p^{(m)} | u_p^{(m')} \rangle &= \langle \bar{u}_p^{(m)} | \bar{u}_p^{(m')} \rangle = 0, \text{ for } m \neq m'.
\end{aligned}$$

In terms of E_p that $E_p \rightarrow 0$, according to the definition of limitation, utilizing small value $\varepsilon \rightarrow 0$ can express the above equations that $\sqrt{\langle v_p^{(m)} | v_p^{(m)} \rangle} = 1 + \varepsilon$ and $\langle \tilde{u}_p^{(m)} | u_p^{(m')} \rangle = \varepsilon$. Then, by introducing the error matrix $E_r = (\varepsilon_{m,m'})_{2^n \times 2^n}$ generally respecting tiny values rather than specific variables, the equations are written as

$$\begin{aligned}
R_p^{-1} &= I + E_r, \\
R_p &= I - E_r (I + E_r)^{-1},
\end{aligned} \tag{b.10}$$

where E_r and $E_r (I + E_r)^{-1}$ are upper triangular matrices with real diagonal values. It implicates that $E_r \rightarrow O$ as well as $E_r (I + E_r)^{-1} \rightarrow O$ and $\varepsilon_{m,m} \in \mathbb{R}, \varepsilon_{m,m'} \in \mathbb{C} (m \neq m')$. Considering that

$Q_p = U_p R_p^{-1}$ where $Q_p = \left[|q_p^{(1)}\rangle, |q_p^{(2)}\rangle, \dots, |q_p^{(2^n)}\rangle \right]$, linear operation of each column is signified as

$$|q_p^{(m)}\rangle = (1 + \varepsilon_{m,m}) |u_p^{(m)}\rangle + \sum_{l=1}^{m-1} \varepsilon_{l,m} |u_p^{(l)}\rangle. \quad (\text{b.11})$$

Next, take the real part of the above equation that

$$\text{Re} \left(|q_p^{(m)}\rangle \right) = (1 + \varepsilon_{m,m}) \text{Re} \left(|u_p^{(m)}\rangle \right) + \sum_{l=1}^{m-1} \text{Re} \left(\varepsilon_{l,m} |u_p^{(l)}\rangle \right). \quad (\text{b.12})$$

Since, $\text{Re}(\varepsilon_{l,m} |u_p^{(l)}\rangle) \rightarrow 0$ and $\varepsilon_{m,m} \rightarrow 0$, according to Proposition A.2, when the sign operator S acts on $|q_p^{(m)}\rangle$,

$$\begin{aligned} \text{Re} \left(S |q_p^{(m)}\rangle \right) &= S \left(\text{Re} \left(|q_p^{(m)}\rangle \right) \right) \\ &= S \left((1 + \varepsilon_{m,m}) \text{Re} \left(|u_p^{(m)}\rangle \right) + \sum_{l=1}^{m-1} \text{Re} \left(\varepsilon_{l,m} |u_p^{(l)}\rangle \right) \right) \\ &= S \left((1 + \varepsilon_{m,m}) \text{Re} \left(|u_p^{(m)}\rangle \right) \right) \\ &= S \left(\text{Re} \left(|u_p^{(m)}\rangle \right) \right) \\ &= \text{Re} \left(S |u_p^{(m)}\rangle \right), \end{aligned} \quad (\text{b.13})$$

which manifests $\text{Re}(SQ_p) = \text{Re}(SU_p)$. In this case, $D_p = I$, so $\tilde{U}_p = Q_p$ referred to (34) and (35), which further demonstrates that $\text{Re}(S\tilde{U}_p) = \text{Re}(SU_p)$.

For the other case that $\exists m_1, m_2, \dots, m_l \in \mathbb{N}^* (1 \leq l \leq 2^n, \forall m \in \{m_1, m_2, \dots, m_l\} : r_{m,m} < 0)$, let $R'_p = (r'_{m,m'})_{2^n \times 2^n}$ be another upper triangular matrix with positive diagonal values. There lies a linear transformation between R_p and R'_p that

$$R_p = D_r R'_p,$$

in which

$$\begin{aligned} D_r &= \text{diag}\{d_{1,1}, d_{2,2}, \dots, d_{2^n, 2^n}\}, \\ d_{m,m} &= \begin{cases} -1, & m \in \{m_1, m_2, \dots, m_l\} \\ 1, & m \notin \{m_1, m_2, \dots, m_l\} \end{cases}. \end{aligned}$$

Therefore, according to Equations (34) and (35), the equation $D_p = D_r$ is concluded and

$$U_p = Q_p R_p = \tilde{U}_p D_r^{-1} D_r R'_p = \tilde{U}_p R'_p,$$

which naturally implicates the first case discussed.

Consequently, the second case can be equivalently converted into the first case so that the propositions that $\lim_{E_p \rightarrow 0} \tilde{U}_p - U_p = O$ and $\text{Re}(S(\tilde{U}_p)) = \text{Re}(S(U_p))$ still hold. $\square$

ACKNOWLEDGMENTS

The authors acknowledge the editor and anonymous reviewers for their valuable suggestions.

REFERENCES

- [1] Rui Abreu, Peter Zoetewij, and Arjan J. C. Van Gemund. 2007. On the accuracy of spectrum-based fault localization. In *Proceedings of the Testing: Academic and Industrial Conference Practice and Research Techniques-MUTATION (TAICPART-MUTATION'07)*. IEEE, 89–98.
- [2] Gadi Aleksandrowicz, Thomas Alexander, Panagiotis Barkoutsos, Luciano Bello, Yael Ben-Haim, David Bucher, Francisco Jose Cabrera-Hernández, Jorge Carballo-Franquis, Adrian Chen, Chun-Fu Chen, Jerry M. Chow, Antonio D. Córcoles-Gonzales, Abigail J. Cross, Andrew Cross, Juan Cruz-Benito, Chris Culver, Salvador De La Puente González, Enrique De La Torre, Delton Ding, Eugene Dumitrescu, Ivan Duran, Pieter Eendebak, Mark Everitt, Ismael Faro Sertage, Albert Frisch, Andreas Fuhrer, Jay Gambetta, Borja Godoy Gago, Juan Gomez-Mosquera, Donny Greenberg, Ikko Hamamura, Vojtech Havlicek, Joe Hellmers, Lukasz Herok, Hiroshi Horii, Shaohan Hu, Takashi Imamichi, Toshinari Itoko, Ali Javadi-Abhari, Naoki Kanazawa, Anton Karazeev, Kevin Krsulich, Peng Liu, Yang Luh, Yunho Maeng, Manoel Marques, Francisco Jose Martín-Fernández, Douglas T. McClure, David McKay, Srujan Meesala, Antonio Mezzacapo, Nikolaj Moll, Diego Moreda Rodríguez, Giacomo Nannicini, Paul Nation, Pauline Ollitrault, Lee James O'Riordan, Hanhee Paik, Jesús Pérez, Anna Phan, Marco Pistoia, Viktor Prutyantov, Max Reuter, Julia Rice, Abdón Rodríguez Davila, Raymond Harry Putra Rudy, Mingi Ryu, Ninad Sathaye, Chris Schnabel, Eddie Schoute, Kanav Setia, Yunong Shi, Adenilton Silva, Yukio Siraichi, Seyon Sivarajah, John A. Smolin, Mathias Soeken, Hitomi Takahashi, Ivano Tavernelli, Charles Taylor, Pete Taylour, Kenso Trabing, Matthew Treinish, Wes Turner, Desiree Vogt-Lee, Christophe Vuillot, Jonathan A. Wildstrom, Jessica Wilson, Erick Winston, Christopher Wood, Stephen Wood, Stefan Wörner, Ismail Yunus Akhalwaya, and Christa Zoufal. 2019. Qiskit: An Open-source framework for quantum computing. <https://doi.org/10.5281/zenodo.2562110>
- [3] Shaukat Ali, Paolo Arcaini, Xinyi Wang, and Tao Yue. 2021. Assessing the effectiveness of input and output coverage criteria for testing quantum programs. In *Proceedings of the 14th IEEE Conference on Software Testing, Verification and Validation (ICST'21)*. IEEE, 13–23.
- [4] Kai-Yuan Cai, João W. Cangussu, Raymond A. DeCarlo, and Aditya P. Mathur. 2003. An overview of software cybernetics. In *Proceedings of the 11th Annual International Workshop on Software Technology and Engineering Practice*. IEEE, 77–86.
- [5] Kai-Yuan Cai, Hai Hu, Chang-Hai Jiang, and Feng Ye. 2009. Random testing with dynamically updated test profile. In *Proceedings of the 20th International Symposium on Software Reliability Engineering (ISSRE'09)*. 1–2.
- [6] Kai-Yuan Cai, Kishor S. Trivedi, and Beibei Yin. 2021. S-ada: Software as an autonomous, dependable and affordable system. In *Proceedings of the 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S'21)*. IEEE, 17–18.
- [7] João W. Cangussu, Kai-Yuan Cai, Scott D. Miller, and Aditya P. Mathur. 2007. Software cybernetics. In *Wiley Encyclopedia of Computer Science and Engineering*.
- [8] Junjie Chen, Yihua Liang, Qingchao Shen, Jiajun Jiang, and Shuochuan Li. 2022. Toward understanding deep learning framework bugs. *ACM Trans. Softw. Eng. Methodol.* 32, 6 (2022), 1–31.
- [9] Mike Y. Chen, Emre Kiciman, Eugene Fratkin, Armando Fox, and Eric Brewer. 2002. Pinpoint: Problem determination in large, dynamic internet services. In *Proceedings International Conference on Dependable Systems and Networks*. IEEE, 595–604.
- [10] Tsong Yueh Chen, Hing Leung, and Ieng Kei Mak. 2005. Adaptive random testing. In *Proceedings of the Advances in Computer Science-ASIAN 2004. Higher-Level Decision Making: 9th Asian Computing Science Conference*. Springer, 320–329.
- [11] Valentin Dallmeier, Andreas Zeller, and Bertrand Meyer. 2009. Generating fixes from object behavior anomalies. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 550–554.
- [12] Favio DeMarco, Jifeng Xuan, Daniel Le Berre, and Martin Monperrus. 2014. Automatic repair of buggy if conditions and missing preconditions with smt. In *Proceedings of the 6th International Workshop on Constraints in Software Testing, Verification, and Analysis*. 30–39.
- [13] Joe W. Duran and Simeon C. Ntafos. 1984. An evaluation of random testing. *IEEE Trans. Softw. Eng.* 4 (1984), 438–444.
- [14] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. 2001. Bugs as deviant behavior: A general approach to inferring errors in systems code. *ACM SIGOPS Operat. Syst. Rev.* 35, 5 (2001), 57–72.
- [15] Richard P. Feynman. 1982. Simulating physics with computers. *Int. J. Theor. Phys.* 21, 6/7 (1982).
- [16] Daniel Fortunato, Jose Campos, and Rui Abreu. 2022. Mutation testing of quantum programs: A case study with Qiskit. *IEEE Trans. Quant. Eng.* 3 (2022), 1–17.
- [17] Daniel Fortunato, José Campos, and Rui Abreu. 2022. Mutation testing of quantum programs written in Qiskit. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 358–359.
- [18] Daniel Fortunato, José Campos, and Rui Abreu. 2022. QMutPy: A mutation testing tool for quantum algorithms and applications in Qiskit. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 797–800.

- [19] Antonio García de la Barrera, Ignacio García-Rodríguez de Guzmán, Macario Polo, and Mario Piattini. 2023. Quantum software testing: State of the art. *J. Software: Evolution and Process* 35, 4 (2023), e2419.
- [20] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2018. Automatic software repair: A survey. In *Proceedings of the 40th International Conference on Software Engineering*. 1219–1219.
- [21] Xiaoyu Guo, Jianjun Zhao, and Pengzhan Zhao. 2024. On repairing quantum programs using ChatGPT. arXiv:2401.14913. Retrieved from <https://arxiv.org/abs/2401.14913>
- [22] Richard Hamlet. 1994. Random testing. *Encycl. Softw. Eng.* 2 (1994), 971–978.
- [23] Shahin Honarvar, Mohammad Reza Mousavi, and Rajagopal Nagarajan. 2020. Property-based testing of quantum programs in Q#. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*. 430–435.
- [24] Zdenek Hradil. 1997. Quantum-state estimation. *Phys. Rev. A* 55, 3 (1997), R1561.
- [25] Rubing Huang, Weifeng Sun, Yinyin Xu, Haibo Chen, Dave Towey, and Xin Xia. 2019. A survey on adaptive random testing. *IEEE Trans. Softw. Eng.* 47, 10 (2019), 2052–2083.
- [26] Yipeng Huang and Margaret Martonosi. 2019. Statistical assertions for validating patterns and finding bugs in quantum programs. In *Proceedings of the 46th International Symposium on Computer Architecture*. 541–553.
- [27] Jiajun Jiang, Yingfei Xiong, Hongyu Zhang, Qing Gao, and Xiangqun Chen. 2018. Shaping program repair space with existing patches and similar code. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 298–309.
- [28] James A. Jones and Mary Jean Harrold. 2005. Empirical evaluation of the tarantula automatic fault-localization technique. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering*. 273–282.
- [29] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the International Symposium on Software Testing and Analysis*. 437–440.
- [30] Claire Le Goues. 2013. *Automatic Program Repair Using Genetic Programming*. Ph.D. Dissertation. University of Virginia.
- [31] Claire Le Goues, Michael Dewey-Vogt, Stephanie Forrest, and Westley Weimer. 2012. A systematic study of automated program repair: Fixing 55 out of 105 bugs for \$8 each. In *Proceedings of the 34th International Conference on Software Engineering (ICSE'12)*. IEEE, 3–13.
- [32] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. Tbar: Revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 31–42.
- [33] Sanjaya Lohani, Thomas A. Searles, Brian T. Kirby, and Ryan T. Glasser. 2021. On the experimental feasibility of quantum state reconstruction via machine learning. *IEEE Trans. Quant. Eng.* 2 (2021), 1–10.
- [34] Fan Long and Martin Rinard. 2015. Staged program repair with condition synthesis. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*. 166–178.
- [35] Fan Long and Martin Rinard. 2016. Automatic patch generation by learning correct code. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 298–312.
- [36] Peixun Long and Jianjun Zhao. 2024. Testing multi-subroutine quantum programs: From unit testing to integration testing. *ACM Trans. Softw. Eng. Methodol.* Just Accepted (April 2024). <https://doi.org/10.1145/3656339>
- [37] Joseph M. Lukens, Kody J. H. Law, Ajay Jasra, and Pavel Lougovski. 2020. A practical and efficient approach for Bayesian quantum state estimation. *New J. Phys.* 22, 6 (2020), 063038.
- [38] Junjie Luo, Pengzhan Zhao, Zhongtao Miao, Shuhan Lan, and Jianjun Zhao. 2022. A comprehensive study of bug fixes in quantum programs. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER'22)*. IEEE, 1239–1246.
- [39] Junpeng Lv, Bei-Bei Yin, and Kai-Yuan Cai. 2014. On the asymptotic behavior of adaptive testing strategy for software reliability assessment. *IEEE Trans. Softw. Eng.* 40, 4 (2014), 396–412.
- [40] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th International Conference on Software Engineering*. 691–701.
- [41] Eñaut Mendiluze, Shaukat Ali, Paolo Arcaini, and Tao Yue. 2021. Muskit: A mutation analysis tool for quantum software testing. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE'21)*. IEEE, 1266–1270.
- [42] Andriy Miransky and Lei Zhang. 2019. On testing quantum programs. In *Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER'19)*. IEEE, 57–60.
- [43] Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum Computation and Quantum Information*. Cambridge University Press.
- [44] Hanyu Pei, Beibei Yin, Linzhi Huang, and Kai-Yuan Cai. 2023. A dynamic random testing strategy in the context of cloud computing. *Softw. Qual. J.* 31, 1 (2023), 243–277.

- [45] Yu Pei, Carlo A. Furia, Martin Nordio, Yi Wei, Bertrand Meyer, and Andreas Zeller. 2014. Automated fixing of programs with contracts. *IEEE Trans. Softw. Eng.* 40, 5 (2014), 427–449.
- [46] Yu Pei, Yi Wei, Carlo A. Furia, Martin Nordio, and Bertrand Meyer. 2011. Code-based automated program fixing. In *Proceedings of the 26th IEEE/ACZInternational Conference on Automated Software Engineering (ASE’11)*. IEEE, 392–395.
- [47] Mauro Pezzè and Michal Young. 2008. *Software Testing and Analysis: Process, Principles, and Techniques*. John Wiley & Sons.
- [48] Thomas Reps, Thomas Ball, Manuvir Das, and James Larus. 1997. The use of program profiling for software maintenance with applications to the year 2000 problem. In *Proceedings of the 6th European Software Engineering Conference Held Jointly with the 5th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 432–449.
- [49] Naoto Sato and Ryota Katsube. 2023. Locating buggy segments in quantum program debugging. arXiv:2309.04266. Retrieved from <https://arxiv.org/abs/2309.04266>
- [50] Chang-ai Sun, Hepeng Dai, Guan Wang, Dave Towey, Tsong Yueh Chen, and Kai-Yuan Cai. 2019. Dynamic random testing of web services: A methodology and evaluation. *IEEE Trans. Serv. Comput.* 15, 2 (2019), 736–751.
- [51] Jiyuan Wang, Fucheng Ma, and Yu Jiang. 2021. Poster: Fuzz testing of quantum program. In *Proceedings of the 14th IEEE Conference on Software Testing, Verification and Validation (ICST’21)*. IEEE, 466–469.
- [52] Jiyuan Wang, Qian Zhang, Guoqing Harry Xu, and Miryung Kim. 2021. QDiff: Differential testing of quantum software stacks. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE’21)*. IEEE, 692–704.
- [53] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2021. Application of combinatorial testing to quantum programs. In *Proceedings of the IEEE 21st International Conference on Software Quality, Reliability and Security (QRS’21)*. IEEE, 179–188.
- [54] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2022. QuSBT: Search-based testing of quantum programs. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 173–177.
- [55] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2023. QuCAT: A Combinatorial Testing Tool for Quantum Software. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE’23)*. IEEE, 2066–2069.
- [56] Xinyi Wang, Tongxuan Yu, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2022. Mutation-based test generation for quantum programs with multi-objective search. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1345–1353.
- [57] Zan Wang, Jian Gao, Xiang Chen, Hao-jie Fu, and Xiang-Yu Fan. 2018. Automatic program repair techniques: A survey. *Chin. J. Comput.* 41, 3 (2018), 588–610.
- [58] Yi Wei, Yu Pei, Carlo A. Furia, Lucas S. Silva, Stefan Buchholz, Bertrand Meyer, and Andreas Zeller. 2010. Automated fixing of programs with contracts. In *Proceedings of the 19th International Symposium on Software Testing and Analysis*. 61–72.
- [59] Westley Weimer, Zachary P. Fry, and Stephanie Forrest. 2013. Leveraging program equivalence for adaptive program repair: Models and first results. In *Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering (ASE’13)*. IEEE, 356–366.
- [60] Westley Weimer, ThanhVu Nguyen, Claire Le Goues, and Stephanie Forrest. 2009. Automatically finding patches using genetic programming. In *Proceedings of the IEEE 31st International Conference on Software Engineering*. IEEE, 364–374.
- [61] Hongji Yang, Feng Chen, and Suleiman Aliyu. 2017. Modern software cybernetics: New trends. *J. Syst. Softw.* 124 (2017), 169–186.
- [62] Pengzhan Zhao, Zhongtao Miao, Shuhan Lan, and Jianjun Zhao. 2023. Bugs4Q: A benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs. *J. Syst. Softw.* 205 (2023), 111805.
- [63] Pengzhan Zhao, Jianjun Zhao, and Lei Ma. 2021. Identifying bug patterns in quantum programs. In *Proceedings of the IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE’21)*. IEEE, 16–21.
- [64] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 341–353.

Received 8 September 2023; revised 28 March 2024; accepted 11 April 2024